# Competitive Programming

Suffix Arrays and Longest Common Prefix

# Suffix Array

- A data structure that exposes repeated substrings within a long string.

| | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|
| a | b | c | a | a | b | c | c | b | a | a | b |

- First, assume you have an end marker
  - One that sorts before any other character in the string (like a null terminator).

| | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| a | b | c | a | a | b | c | c | b | a | a | b | $ |

$ is a common choice, ASCII code 36.

# Suffix Array

- Consider all the suffixes of this string:

| | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| a | b | c | a | a | b | c | c | b | a | a | b | $ |
| b | c | a | a | b | c | c | b | a | a | b | $ | |
| c | a | a | b | c | c | b | a | a | b | $ | | |
| a | a | b | c | c | b | a | a | b | $ | | | |
| a | b | c | c | b | a | a | b | $ | | | | |
| b | c | c | b | a | a | b | $ | | | | | |
| c | c | b | a | a | b | $ | | | | | | |
| c | b | a | a | b | $ | | | | | | | |
| b | a | a | b | $ | | | | | | | | |
| a | a | b | $ | | | | | | | | | |
| a | b | $ | | | | | | | | | | |
| b | $ | | | | | | | | | | | |
| $ | | | | | | | | | | | | |

# Suffix Array

- Each one identified by its offset.

I start at index 0.

I start at index 1.

| | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 0 | a | b | c | a | a | b | c | c | b | a | a | b | $ |
| 1 | b | c | a | a | b | c | c | b | a | a | b | $ | |
| 2 | c | a | a | b | c | c | b | a | a | b | $ | | |
| 3 | a | a | b | c | c | b | a | a | b | $ | | | |
| 4 | a | b | c | c | b | a | a | b | $ | | | | |
| 5 | b | c | c | b | a | a | b | $ | | | | | |
| 6 | c | c | b | a | a | b | $ | | | | | | |
| 7 | c | b | a | a | b | $ | | | | | | | |
| 8 | b | a | a | b | $ | | | | | | | | |
| 9 | a | a | b | $ | | | | | | | | | |
| 10 | a | b | $ | | | | | | | | | | |
| 11 | b | $ | | | | | | | | | | | |
| 12 | $ | | | | | | | | | | | | |

# Suffix Array

- The suffix array is like sorting all these suffixes lexicographically.
  - We don't really make all these copies of the input string.
  - We just sort the array of offsets.
- You might think you could do this with a sort … but just comparing two strings could cost O( n ) time.

| | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 12 | $ | | | | | | | | | | | | |
| 9 | a | a | b | $ | | | | | | | | | |
| 3 | a | a | b | c | c | b | a | a | b | $ | | | |
| 10 | a | b | $ | | | | | | | | | | |
| 0 | a | b | c | a | a | b | c | c | b | a | a | b | $ |
| 4 | a | b | c | c | b | a | a | b | $ | | | | |
| 11 | b | $ | | | | | | | | | | | |
| 8 | b | a | a | b | $ | | | | | | | | |
| 1 | b | c | a | a | b | c | c | b | a | a | b | $ | |
| 5 | b | c | c | b | a | a | b | $ | | | | | |
| 2 | c | a | a | b | c | c | b | a | a | b | $ | | |
| 7 | c | b | a | a | b | $ | | | | | | | |
| 6 | c | c | b | a | a | b | $ | | | | | | |

# Sorting 2-character keys

- We by choosing a two-character key from the start of each suffix.

| | | | |
|---|---|---|---|
| 0 | a | b | a b c a a b c c b a a b $ |
| 1 | b | c | b c a a b c c b a a b $ |
| 2 | c | a | c a a b c c b a a b $ |
| 3 | a | a | a a b c c b a a b $ |
| 4 | a | b | a b c c b a a b $ |
| 5 | b | c | b c c b a a b $ |
| 6 | c | c | c c b a a b $ |
| 7 | c | b | c b a a b $ |
| 8 | b | a | b a a b $ |
| 9 | a | a | a a b $ |
| 10 | a | b | a b $ |
| 11 | b | $ | b $ |
| 12 | $ | \0 | $ |

# Sorting 2-character keys

- Order the strings by that key.
  - This is O( n log n ), but the sorted order won't be perfect.

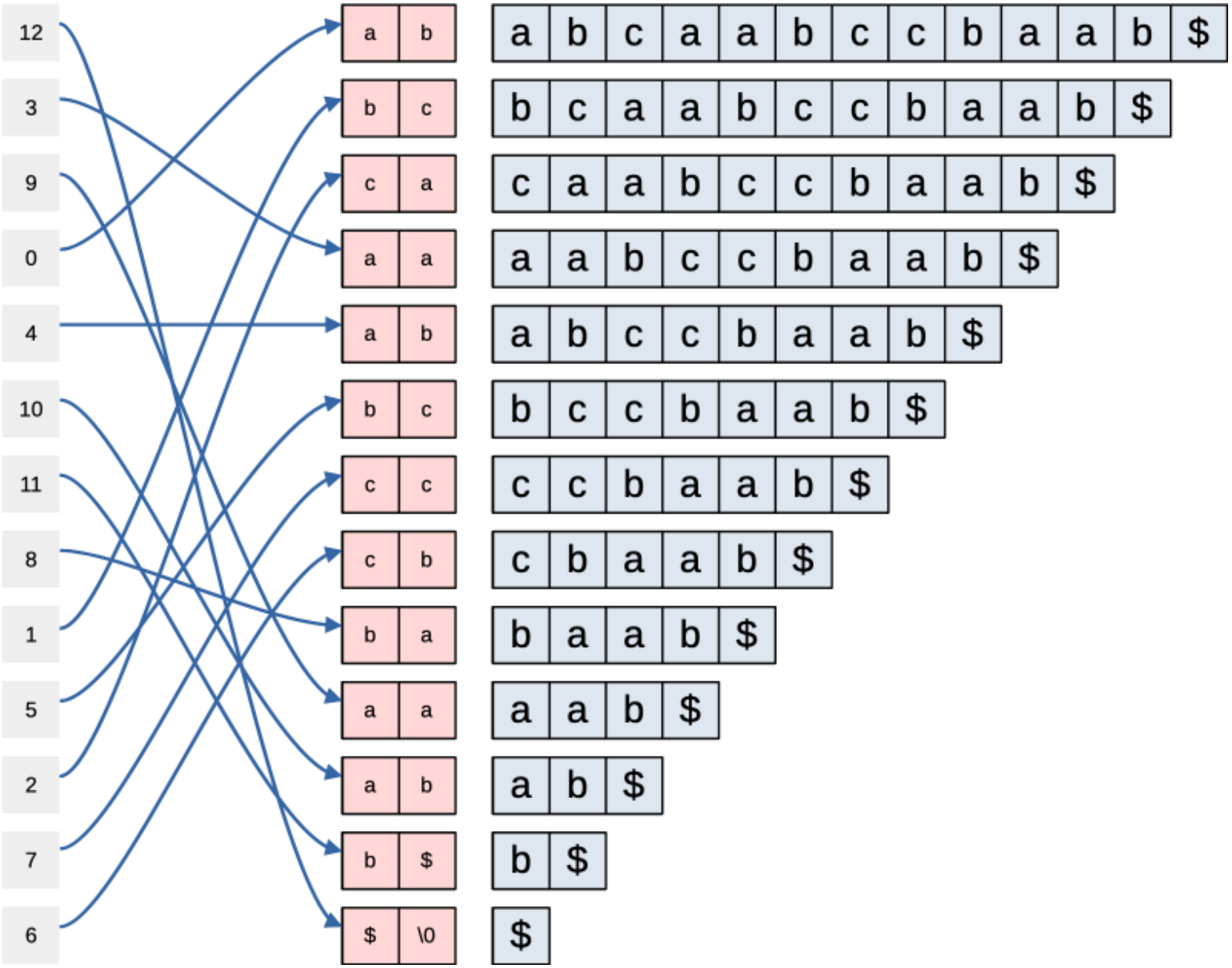
| | | | |
|---|---|---|---|
| 12 | $ | \0 | $ |
| 3 | a | a | a a b c c b a a b $ |
| 9 | a | a | a a b $ |
| 0 | a | b | a b c a a b c c b a a b $ |
| 4 | a | b | a b c c b a a b |
| 10 | a | b | a b $ |
| 11 | b | $ | b $ |
| 8 | b | a | b a a b $ |
| 1 | b | c | b c a a b c c b a a b $ |
| 5 | b | c | b c c b a a b $ |
| 2 | c | a | c a a b c c b a a b $ |
| 7 | c | b | c b a a b $ |
| 6 | c | c | c c b a a b $ |

We're in the right order for just these two characters,

But not for the whole suffix.

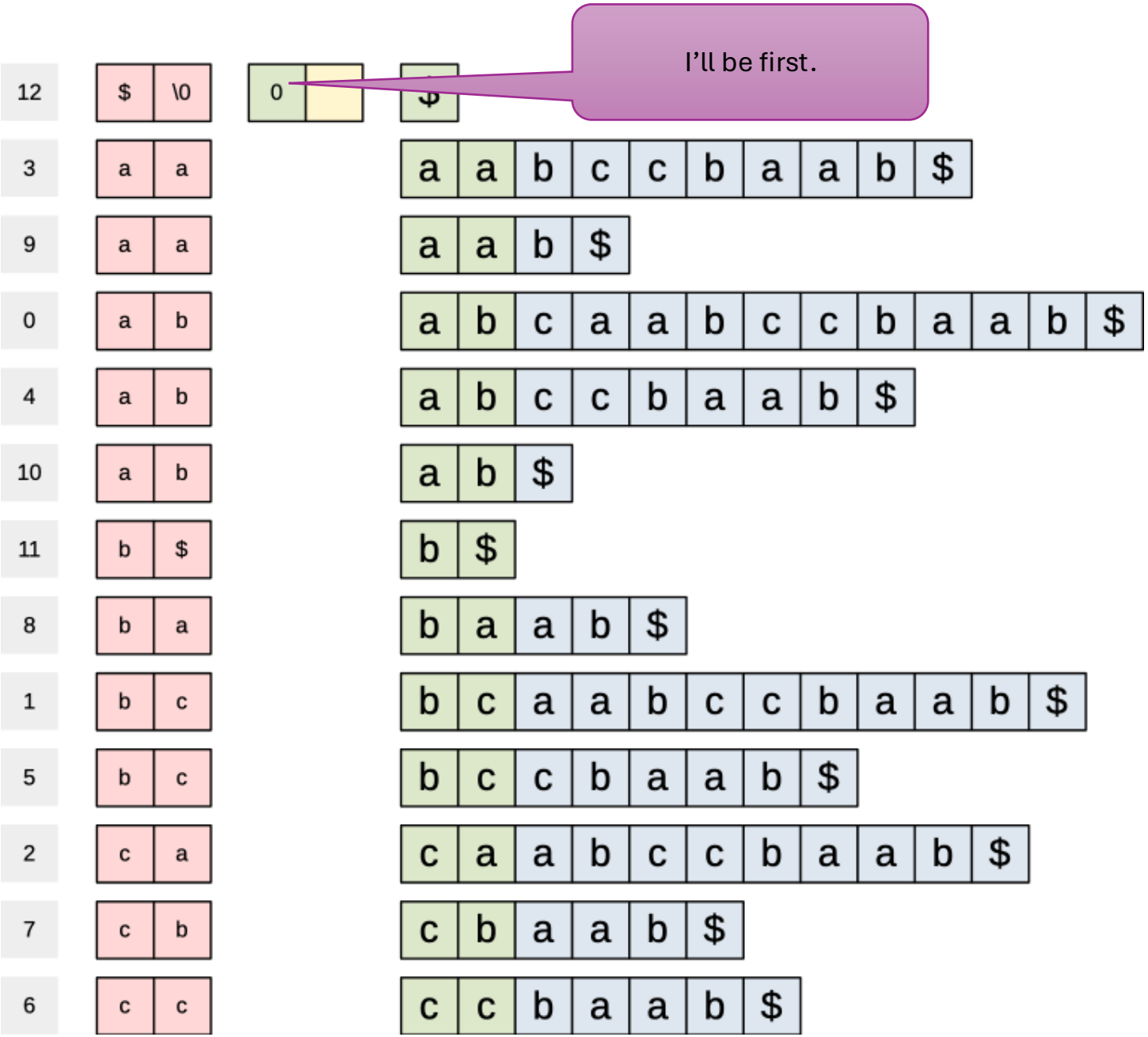
# Sorting Offsets

- We don't really reorder copies of the suffixes.
- We just rearrange an array of offsets.
- Each offset says where we start in the string.



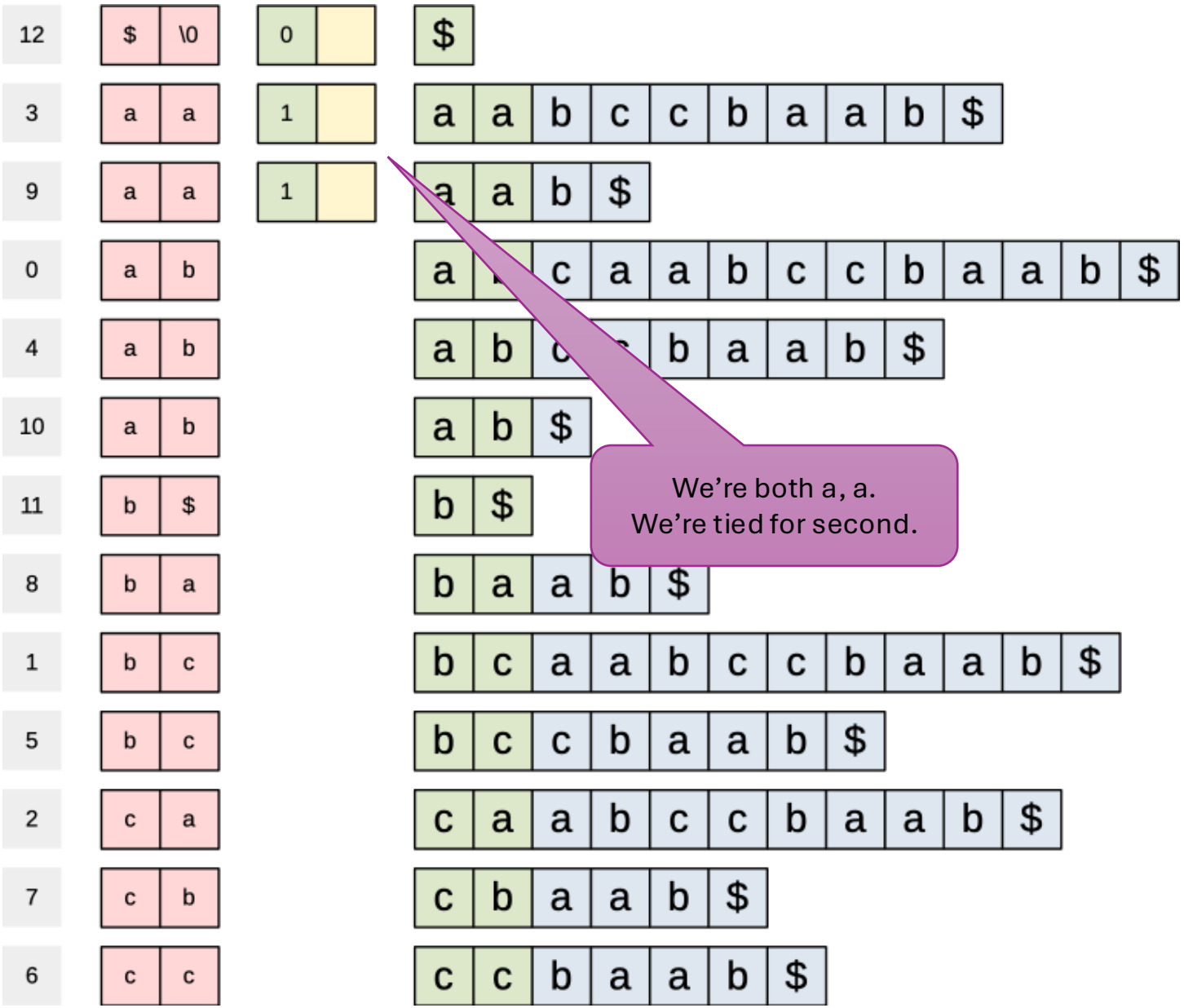
# Extending the Keys

- Ordering of the tells us how the first two characters compare.
- We can pack this ordering into just one integer.



# Extending the Keys

- Ordering of the tells us how the first two characters compare.
- We can pack this ordering into just one integer.



# Extending the Keys

- Ordering of the tells us how the first two characters compare.
- We can pack this ordering into just one integer.

| | | | | |
|---|---|---|---|---|
| 12 | $ | \0 | 0 | $ |
| 3 | a | a | 1 | a a b c c b a a b $ |
| 9 | a | a | 1 | a a b $ |
| 0 | a | b | 2 | a b c a a b c c b a a b $ |
| 4 | a | b | 2 | a b c c b a a b $ |
| 10 | a | b | 2 | a b $ |
| 11 | b | $ | | b $ |
| 8 | b | a | | b a a b $ |
| 1 | b | c | | b c a a b c c b a a b $ |
| 5 | b | c | | b c c b a a b $ |
| 2 | c | a | | c a a b c c b a a b $ |
| 7 | c | b | | c b a a b $ |
| 6 | c | c | | c c b a a b $ |

We're all a,b, we're tied for the next spots.

# Extending the Keys

- Ordering of the tells us how the first two characters compare.
- We can pack this ordering into just one integer.

We could add the next character and sort longer suffixes.

| | | | | | |
|---|---|---|---|---|---|
| 12 | $ | \0 | 0 | | $ |
| 3 | a | a | 1 | | a a b c c b a a b $ |
| 9 | a | a | 1 | | a a b $ |
| 0 | a | b | 2 | | a b c a a b c c b a a b $ |
| 4 | a | b | 2 | | a b c c b a a b $ |
| 10 | a | b | 2 | | a b $ |
| 11 | b | $ | 3 | | b $ |
| 8 | b | a | 4 | | b a a b $ |
| 1 | b | c | 5 | | b c a a b c c b a a b $ |
| 5 | b | c | 5 | | b c c b a a b $ |
| 2 | c | a | 6 | | c a a b c c b a a b $ |
| 7 | c | b | 7 | | c b a a b $ |
| 6 | c | c | 8 | | c c b a a b $ |

# Extending the Keys

- Then, we can find a rank to cover the next two characters

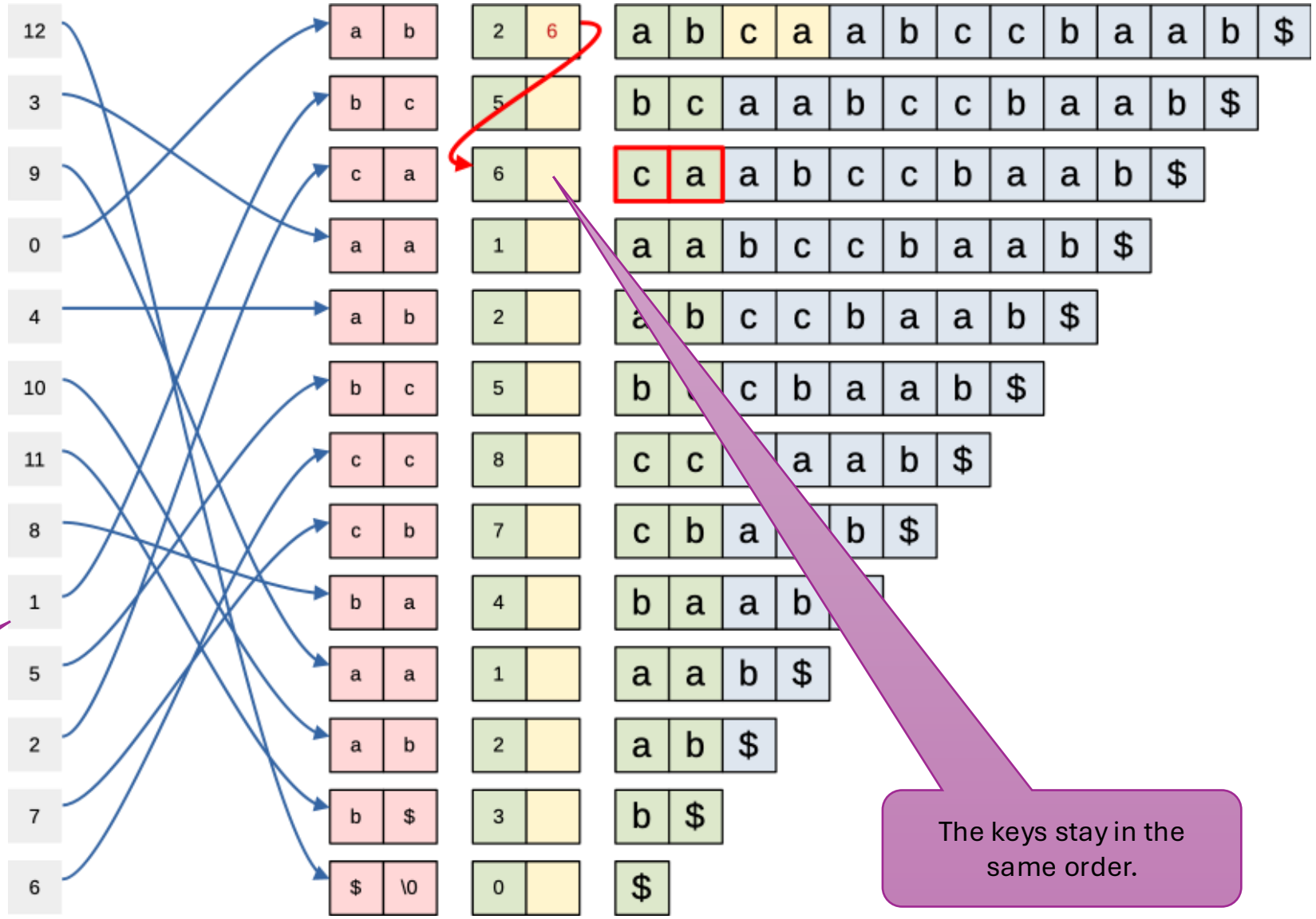
| | | | | | |
|---|---|---|---|---|---|
| 12 | $ | \0 | 0 | | $ |
| 3 | a | a | 1 | | a a b c c b a a b $ |
| 9 | a | a | 1 | | a a b $ |
| 0 | a | b | 2 | 6 | a b c a a b c c b a a b $ |
| 4 | a | b | 2 | | a b c c b a a b $ |
| 10 | a | b | 2 | | a b $ |
| 11 | b | $ | 3 | | b $ |
| 8 | b | a | 4 | | b a a b $ |
| 1 | b | c | 5 | | b c a a b c b a a b $ |
| 5 | b | c | 5 | | b c c b a a b $ |
| 2 | c | a | 6 | | c a a b c c b a a b $ |
| 7 | c | b | 7 | | c b a a b $ |
| 6 | c | c | 8 | | c c b a a b $ |

But, we already have a rank for the next two characters.

How do we efficiently find this?

# Extending the Keys

- Then, we can find a rank to cover the next two characters

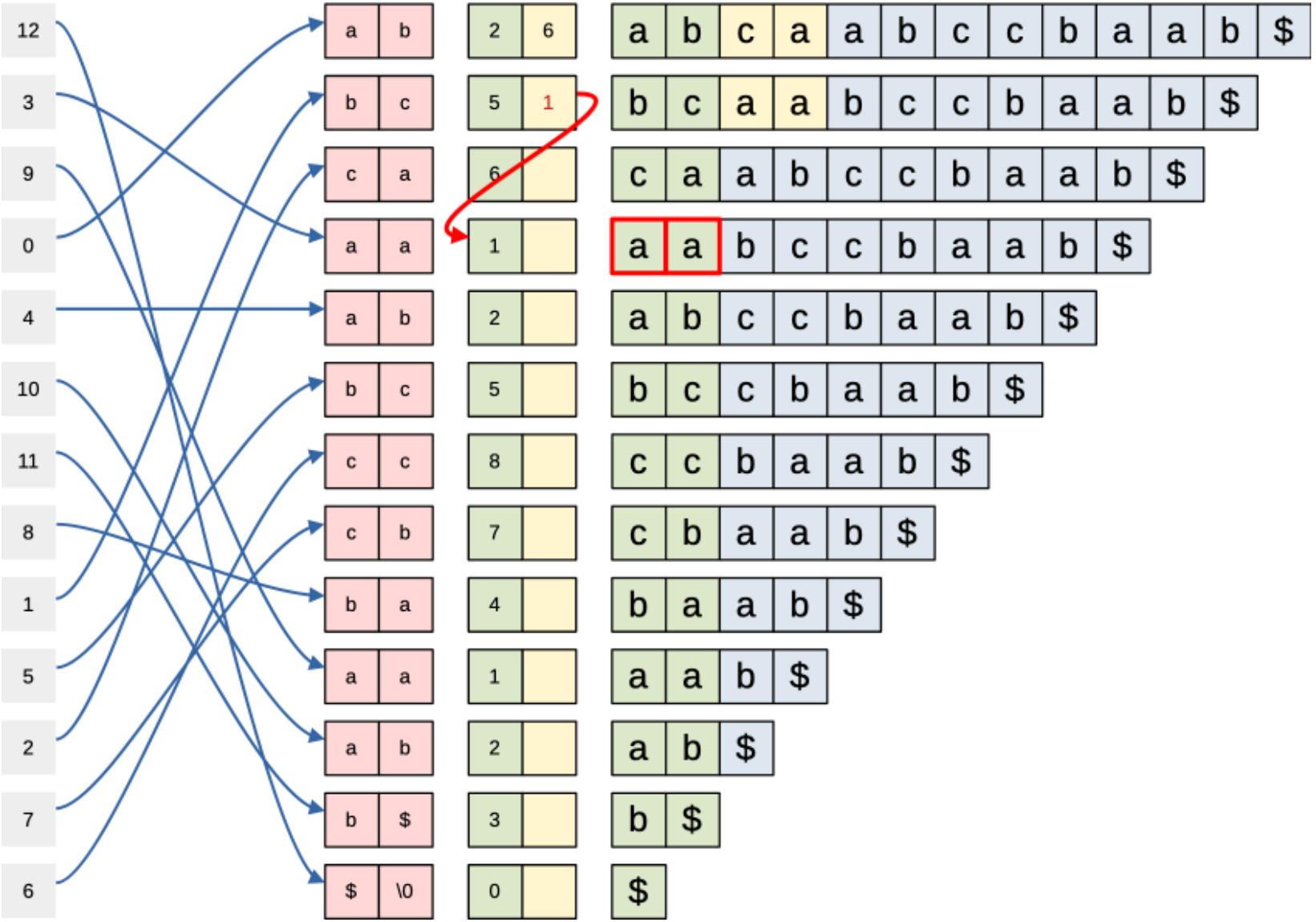



Remember, only the offsets get sorted.

The keys stay in the same order.

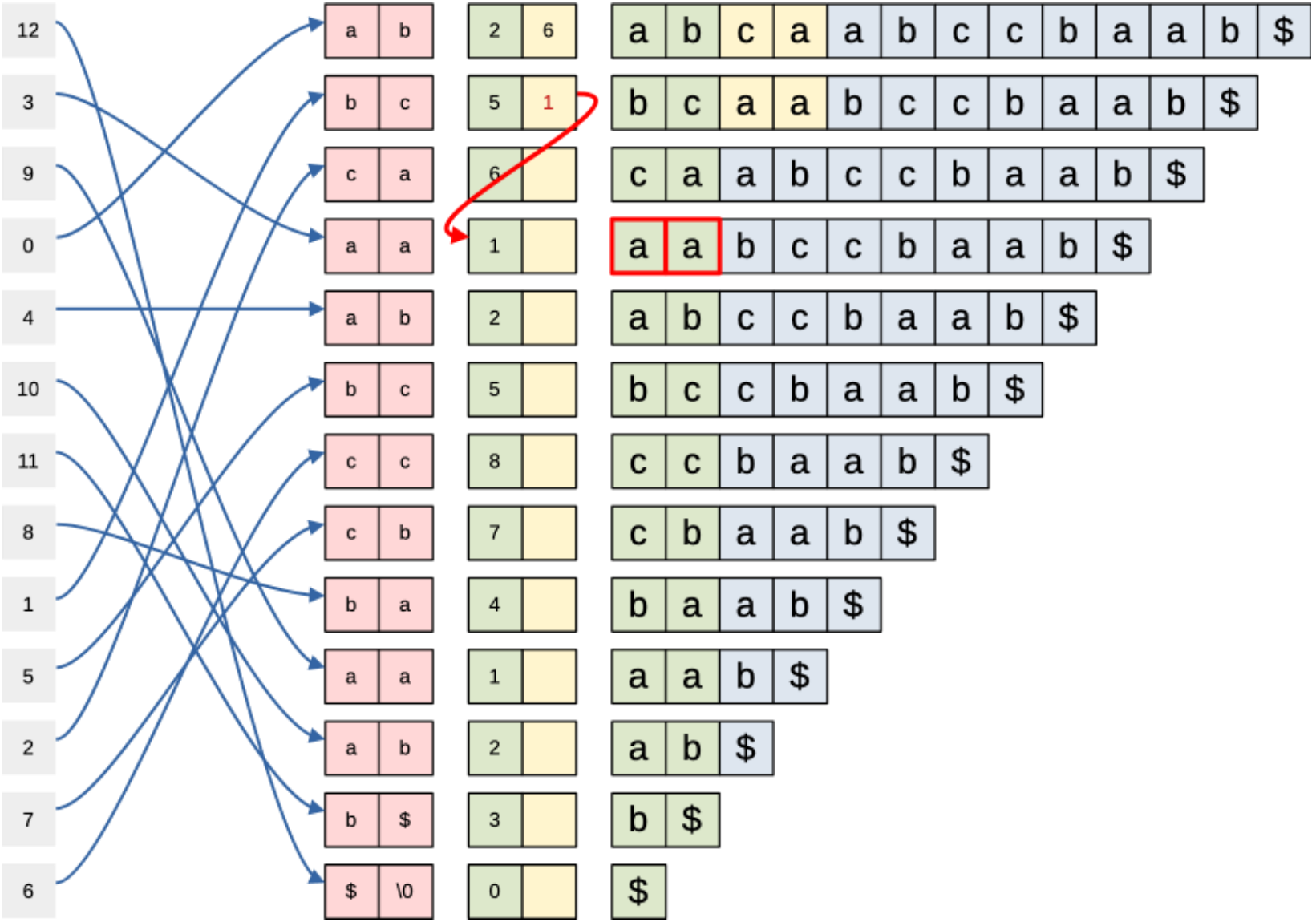
# Extending the Keys

- Then, we can find a rank to cover the next two characters



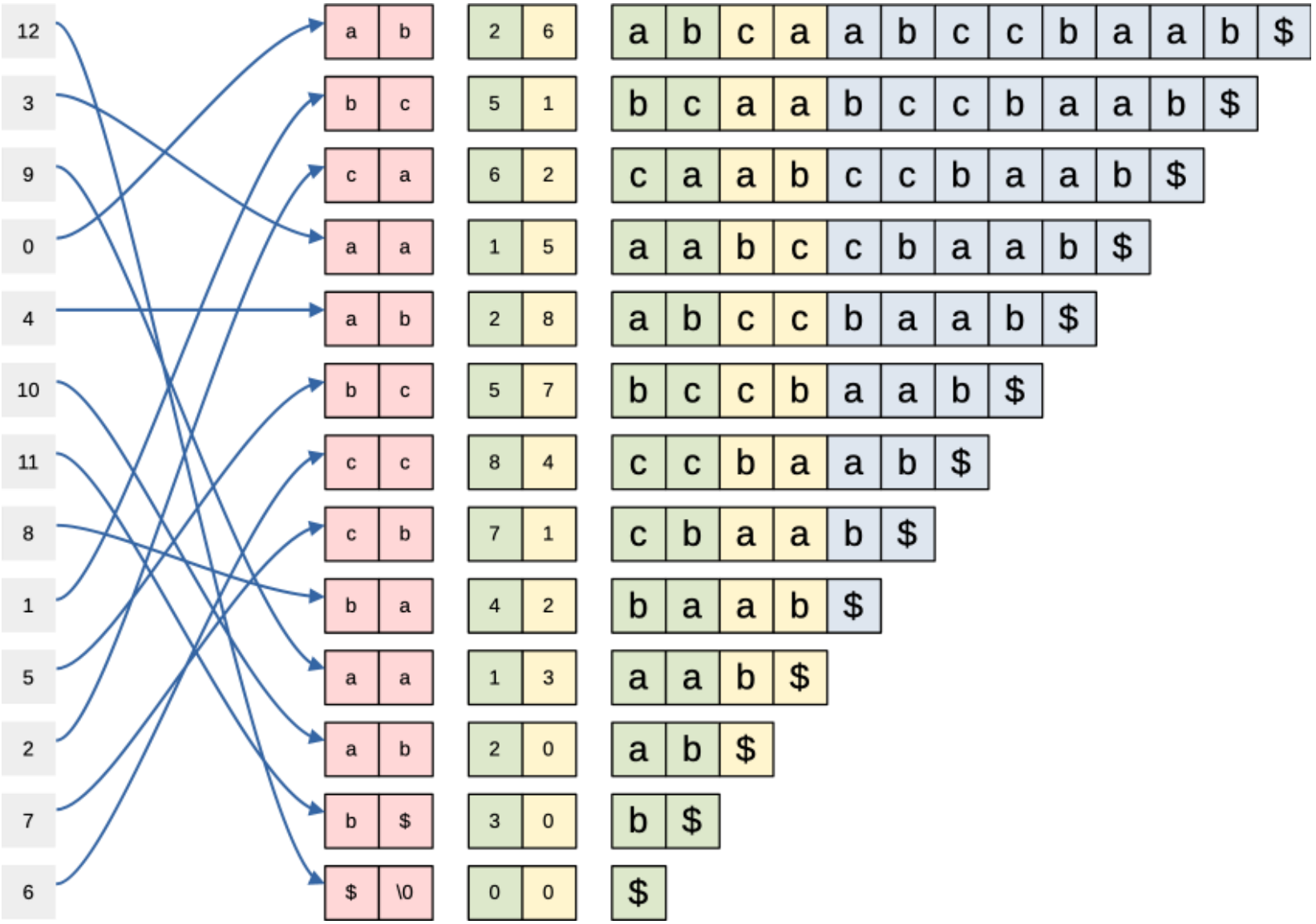
# Extending the Keys

- Then, we can find a rank to cover the next two characters.
- Let's try filling in all the yellow boxes.



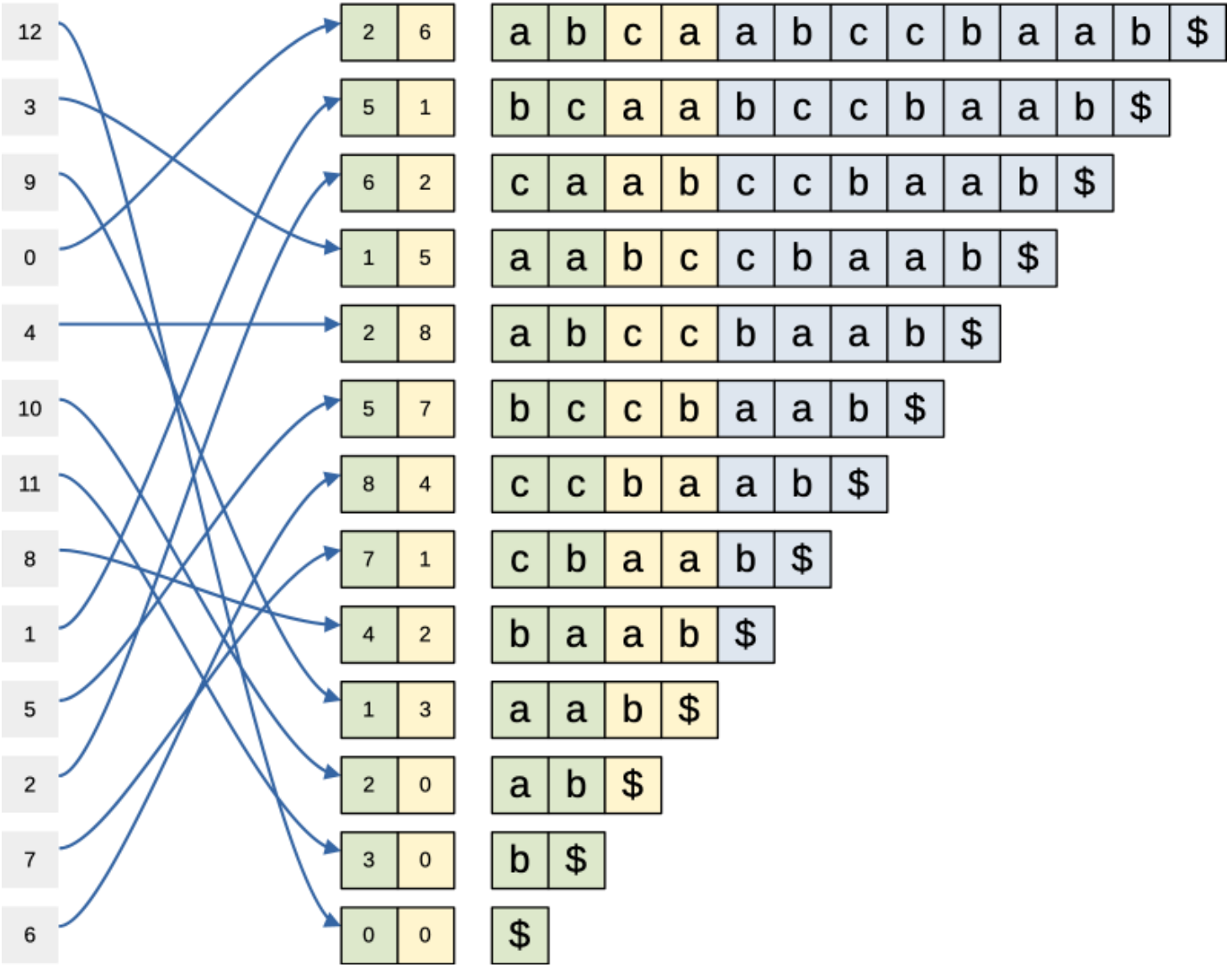
# Extending the Keys

- Then, we can find a rank to cover the next two characters.
- Let's try filling in all the yellow boxes.
- Here's what we should get.



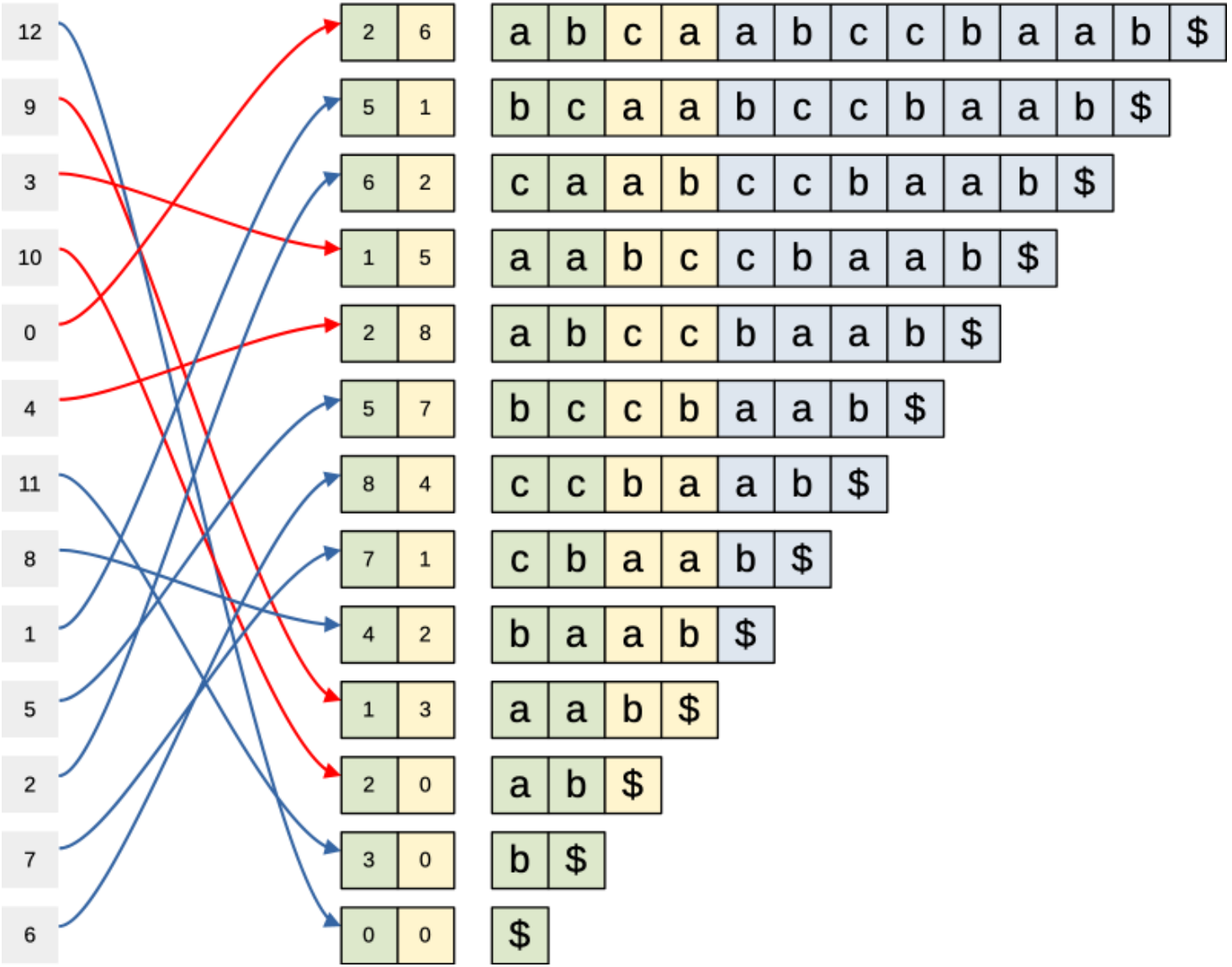
# Extending the Keys

- Now, we have keys representing the first 4 characters.



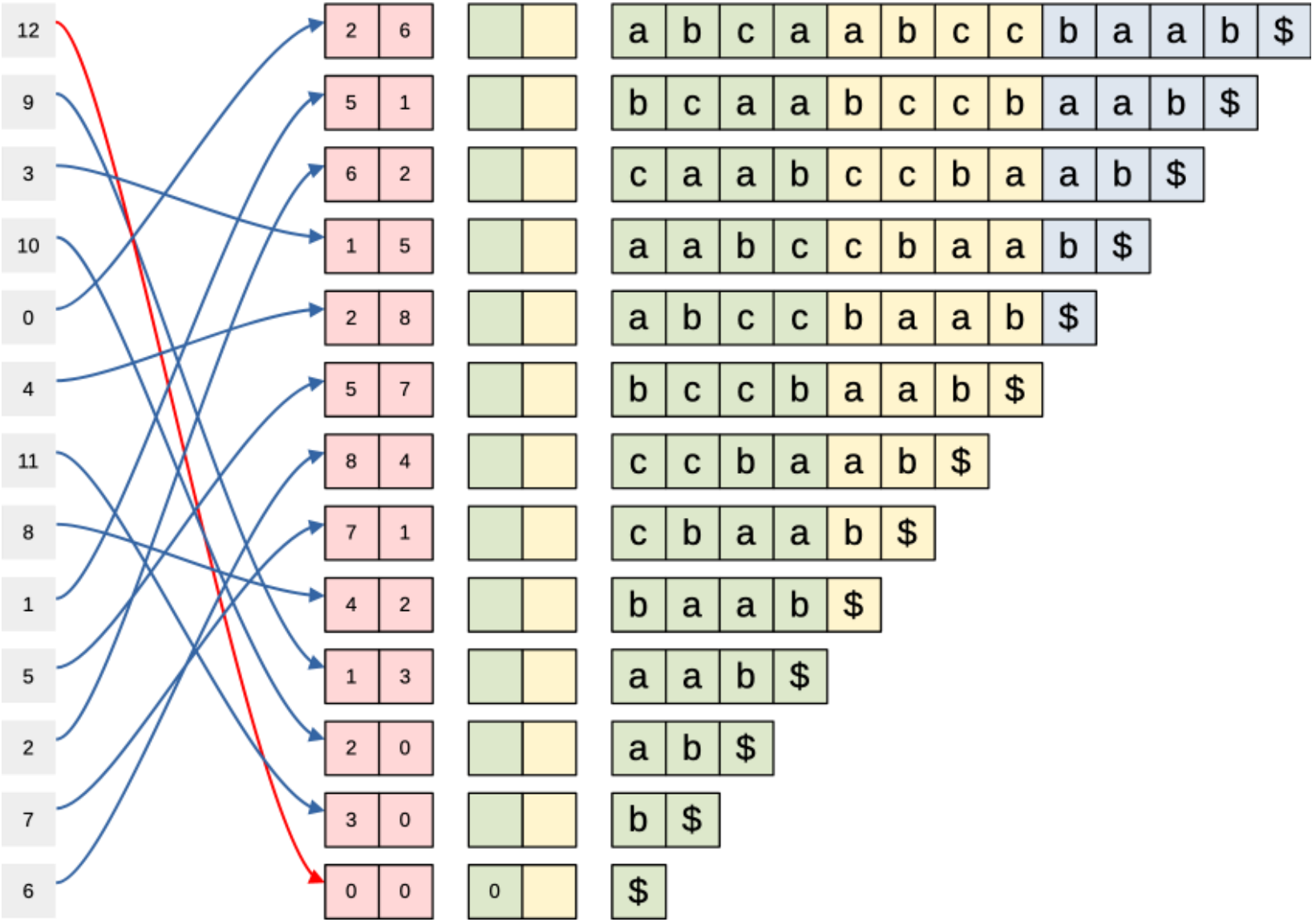
# Sorting Again

- We can sort by those in O( n log n ) time.



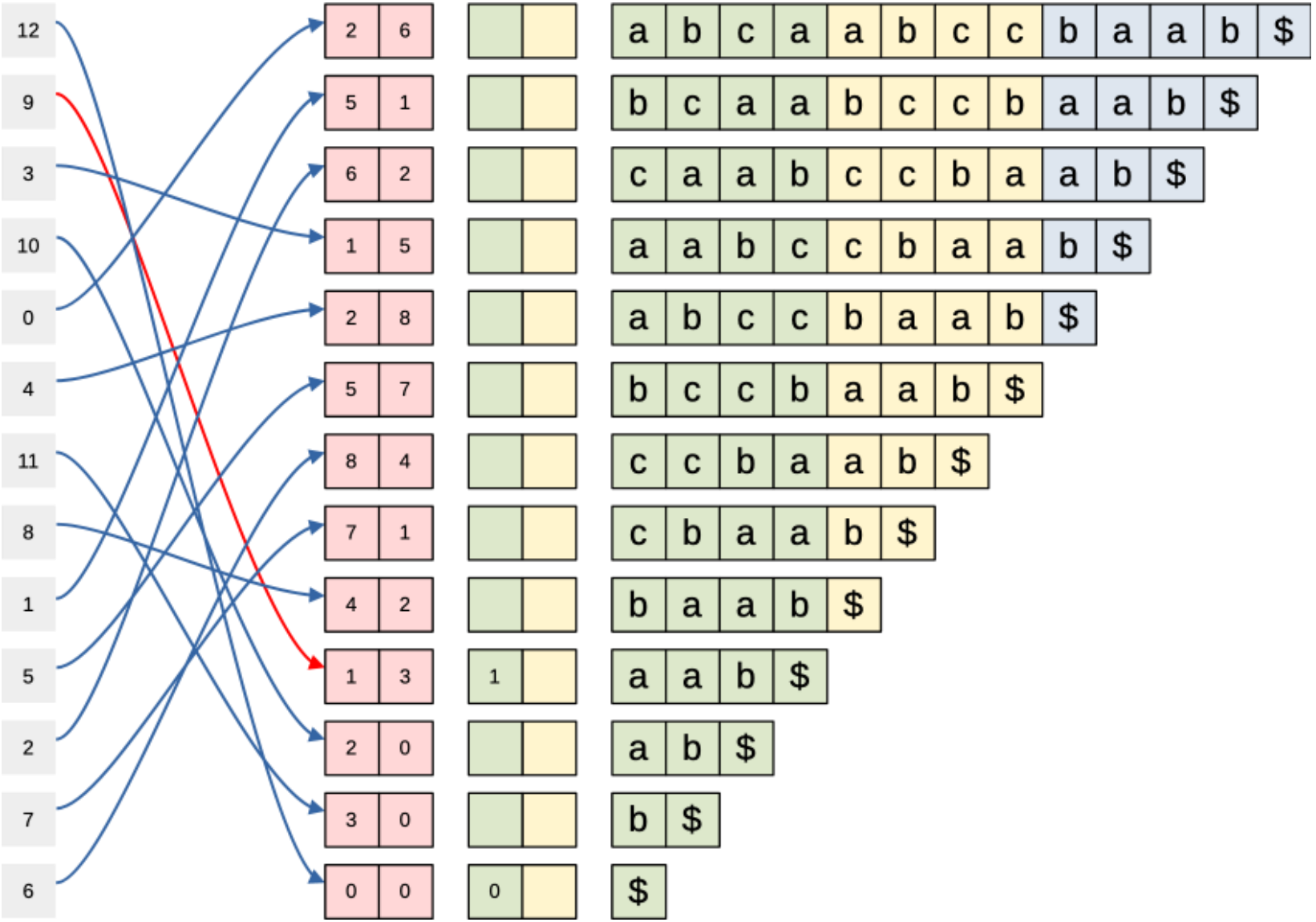
# Extending the Keys

- Now, we can rank these keys representing 4-character sequences.



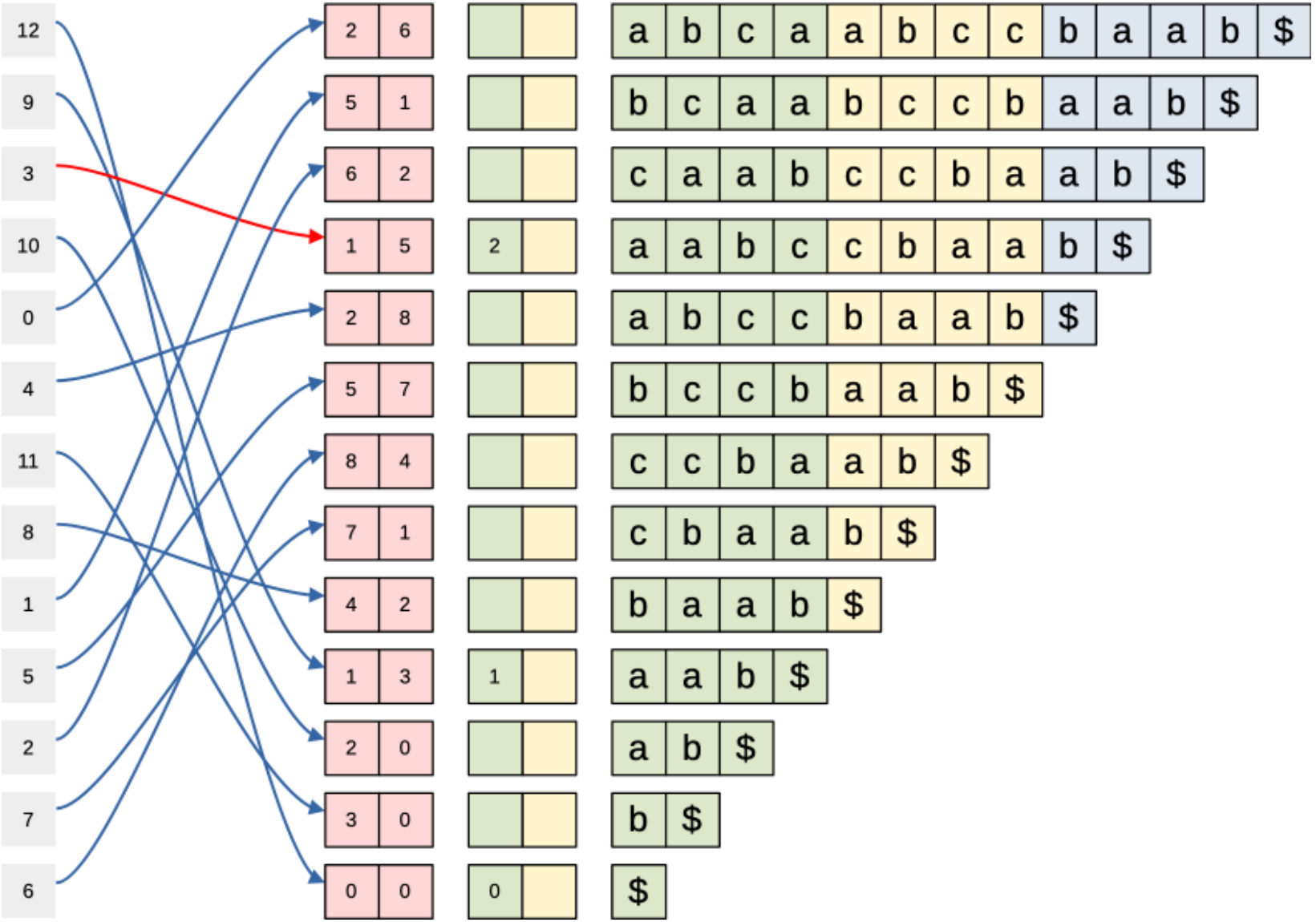
# Extending the Keys

- Now, we can rank these keys representing 4-character sequences.



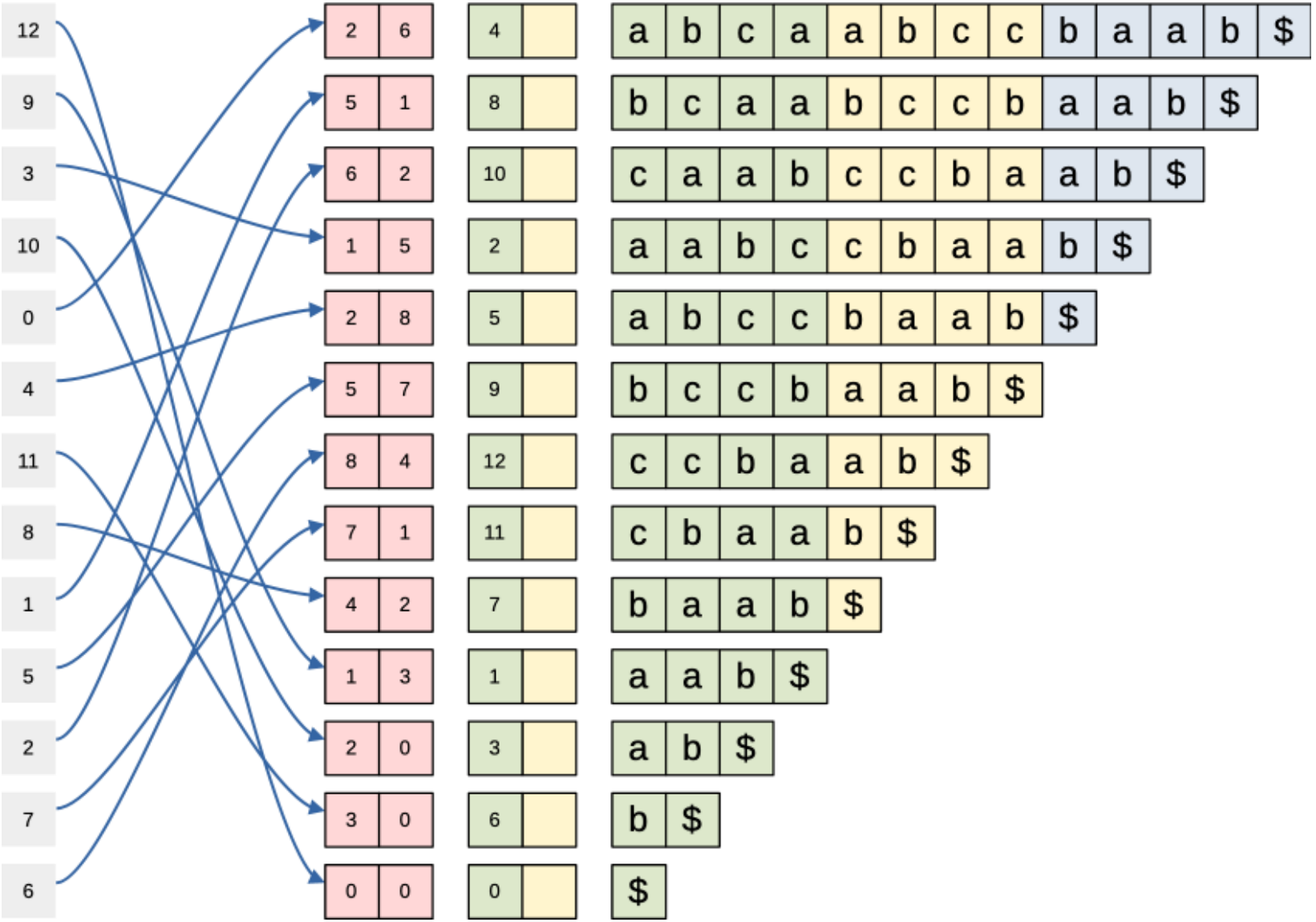
# Extending the Keys

- Now, we can rank these keys representing 4-character sequences.



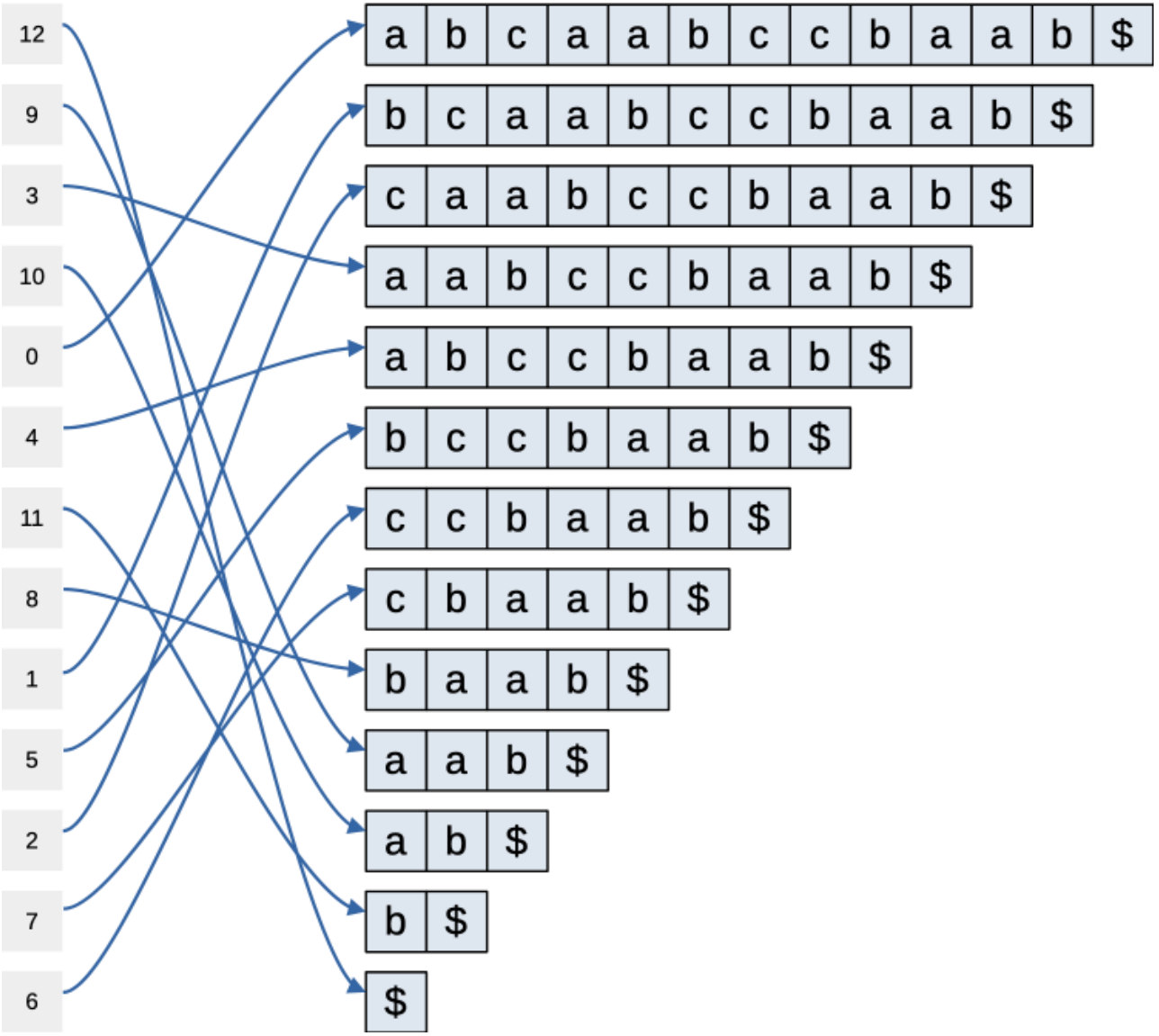
# Extending the Keys

- We could continue, but every string has a unique rank.
- All the offsets are in the right order.



# The Suffix Array

- The array of offsets is our suffix array.
- What's it good for?
  - All kinds of things.
  - All the substrings show up in sorted order.
  - We could binary search for a substring.
  - We could count occurrences of a substring.



# Implementing Suffix Array

```
void makeSuffixArray( string &str, vector< int > &sa ) {
  str += '$';
  int n = str.length();

  sa.assign( n, 0 );
  rnk.assign( n, pair< int, int >( 0, 0 ) );
  for ( int i = 0; i < n; i++ ) {
    rnk[ i ].first = str[ i ];
    sa[ i ] = i;
  }

  …
```

Add the end marker.

String characters as our initial key.

# Implementing Suffix Array

```
vector< pair< int, int > > tempRnk( n );
for ( int k = 1; k < n; k *= 2 ) {
  for ( int i = 0; i < n; i++ )
    rnk[i].second = i + k < n ? rnk[i+k].first : 0;

  sort sa based on rnk[ sa[ i ] ];

  int r = 0;
  tempRnk[ sa[0] ] = pair< int, int >( r, 0 );
  for ( int i = 1; i < n; i++ )
    if ( rnk[sa[i]] == rnk[sa[i-1]] )
      tempRnk[sa[i]] = pair< int, int >( r, 0 );
    else
      tempRnk[sa[i]] = pair< int, int >( ++r, 0 );

  rnk = tempRnk;
  if ( r == n - 1 ) {
    str.pop_back();
    return;
  }
}
```

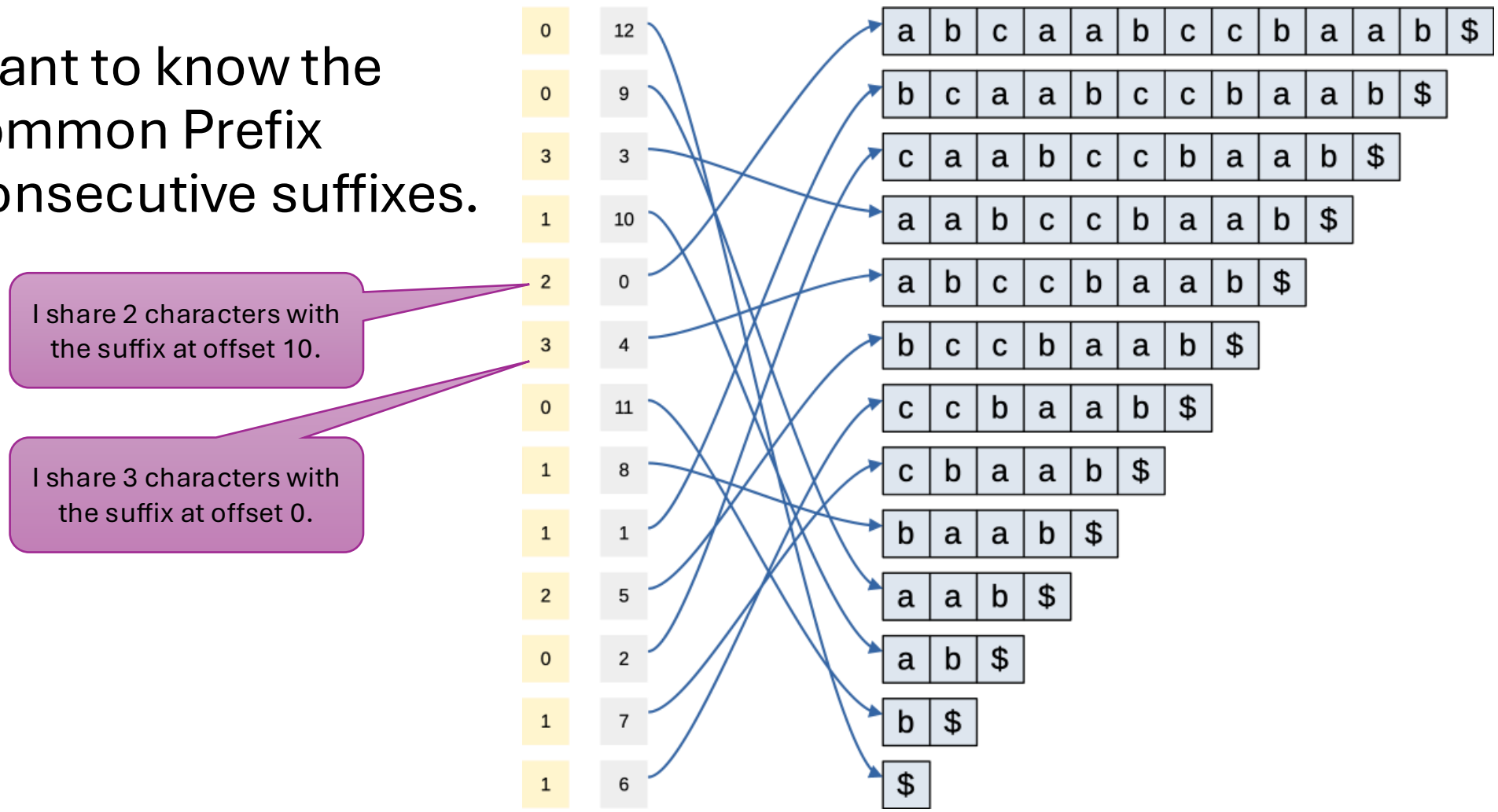
For longer and longer strings

Fill in the second part of the key.

Give out the same rank when we have a tie.

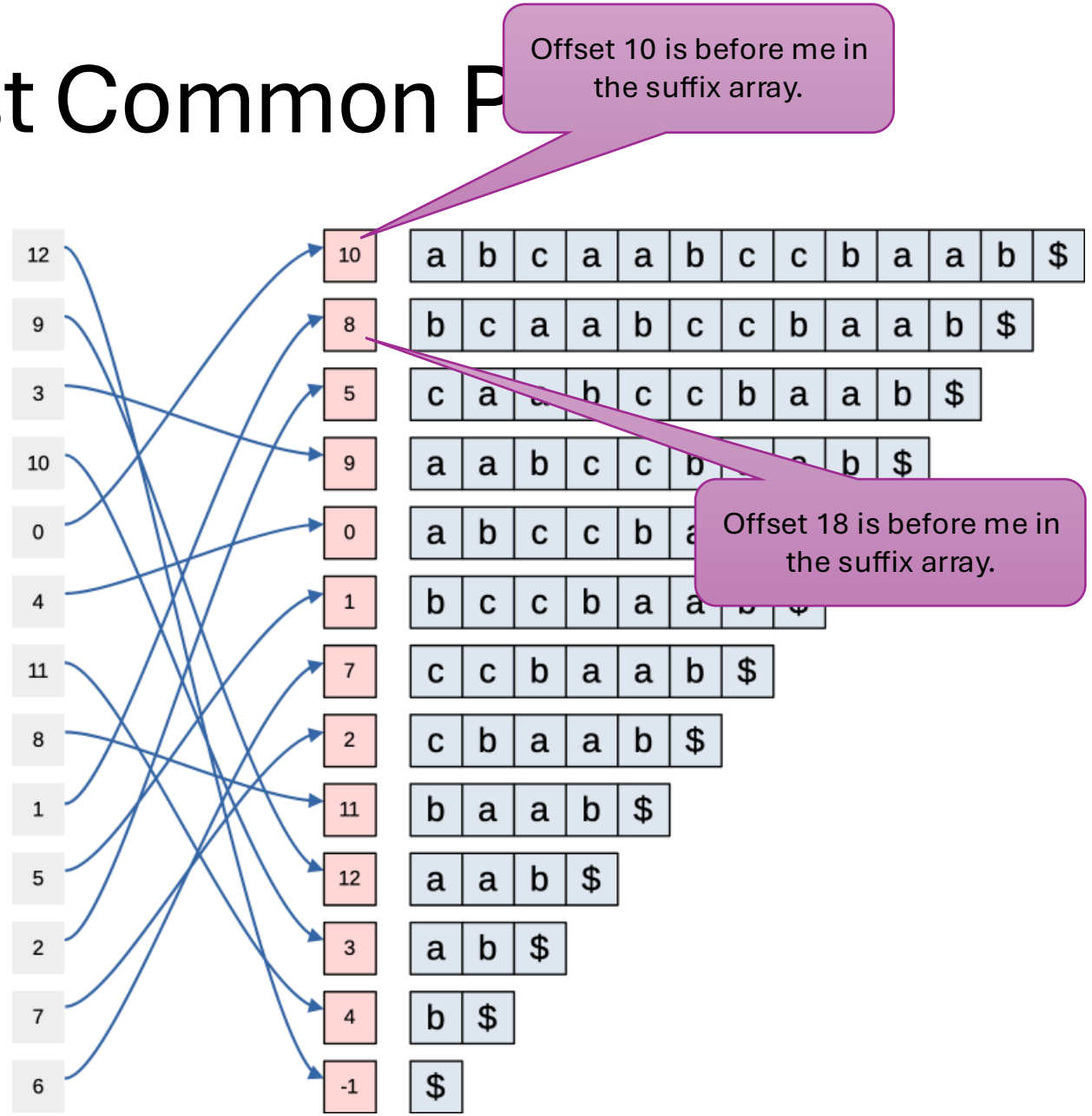
# Computing the Longest Common Prefix

- May also want to know the Longest Common Prefix between consecutive suffixes.



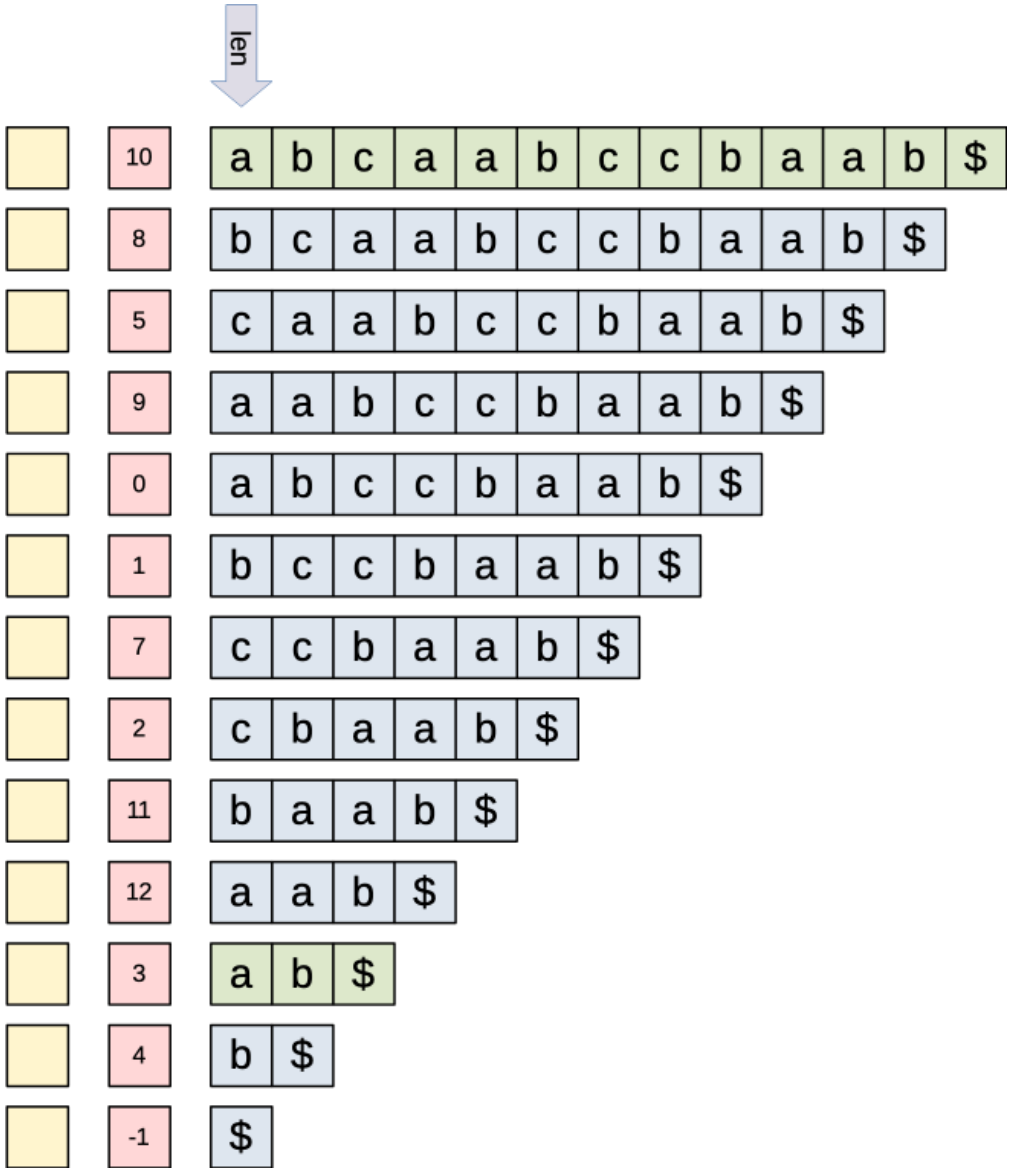
# Computing the Longest Common P

- First, compute the offset of each string's predecessor.



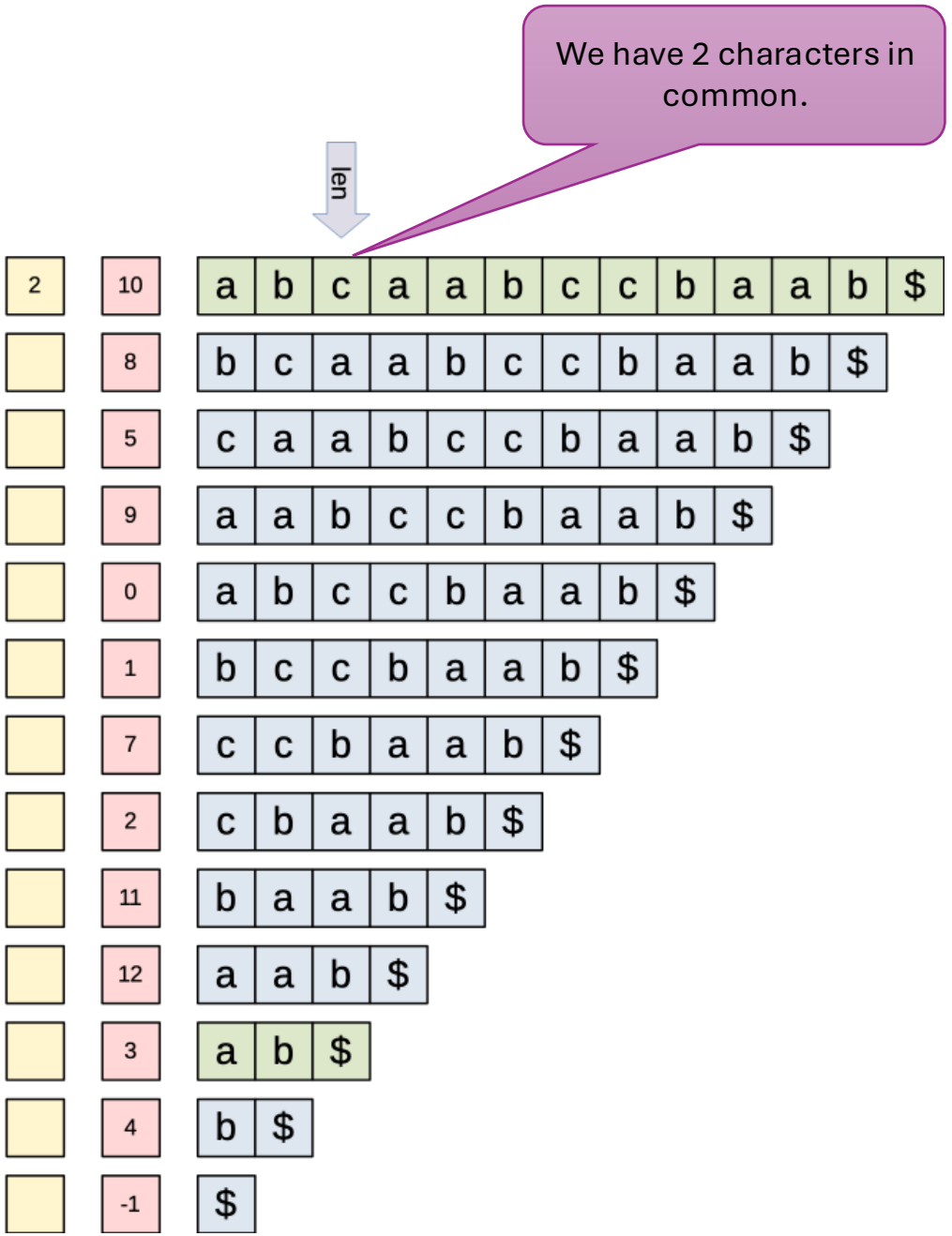
# Computing the Longest Common Prefix

- Count common characters between each string and its predecessor.



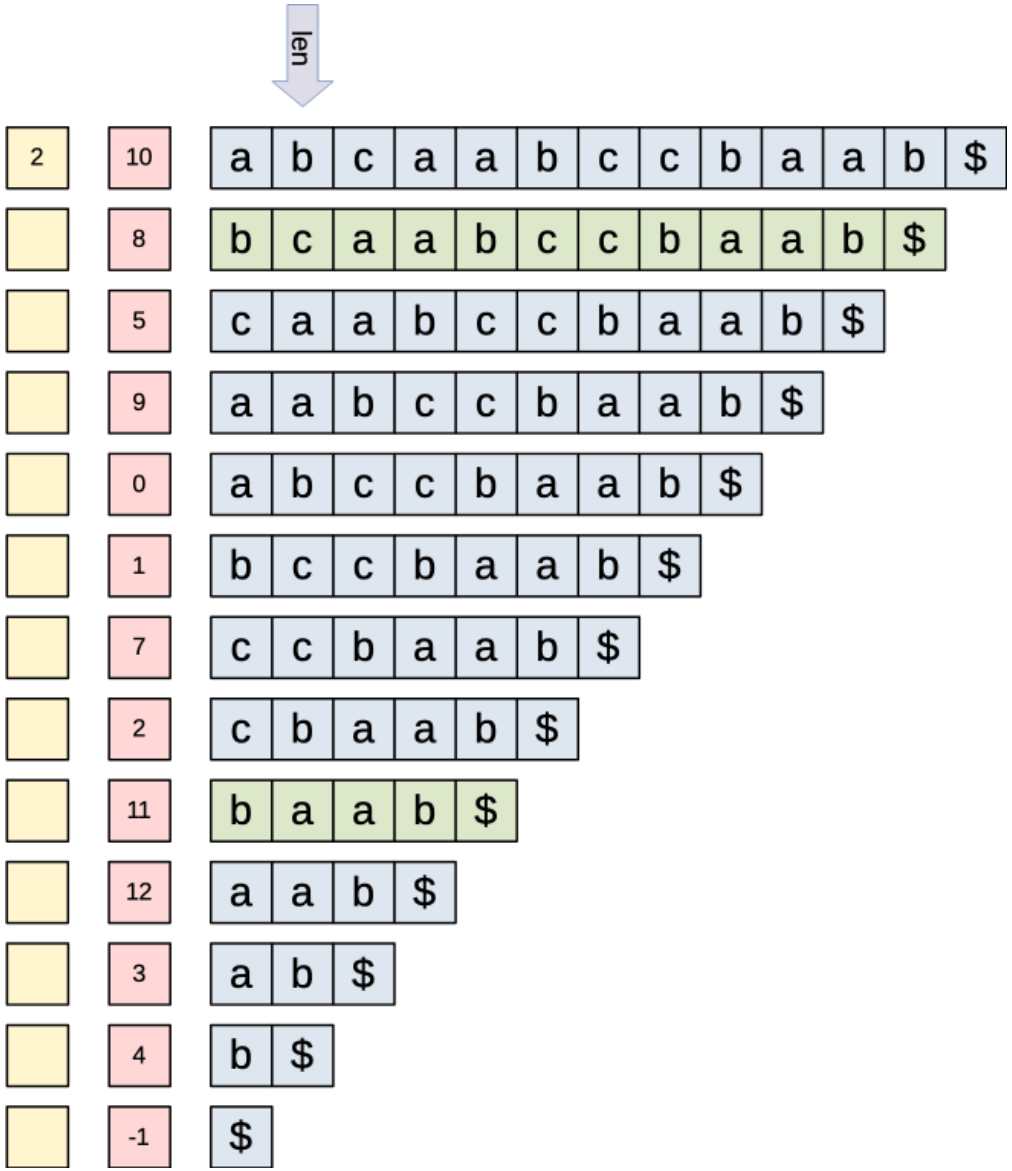
# Computing the Longest Common Prefix

- Count common characters between each string and its predecessor.



# Computing the Longest Common Prefix

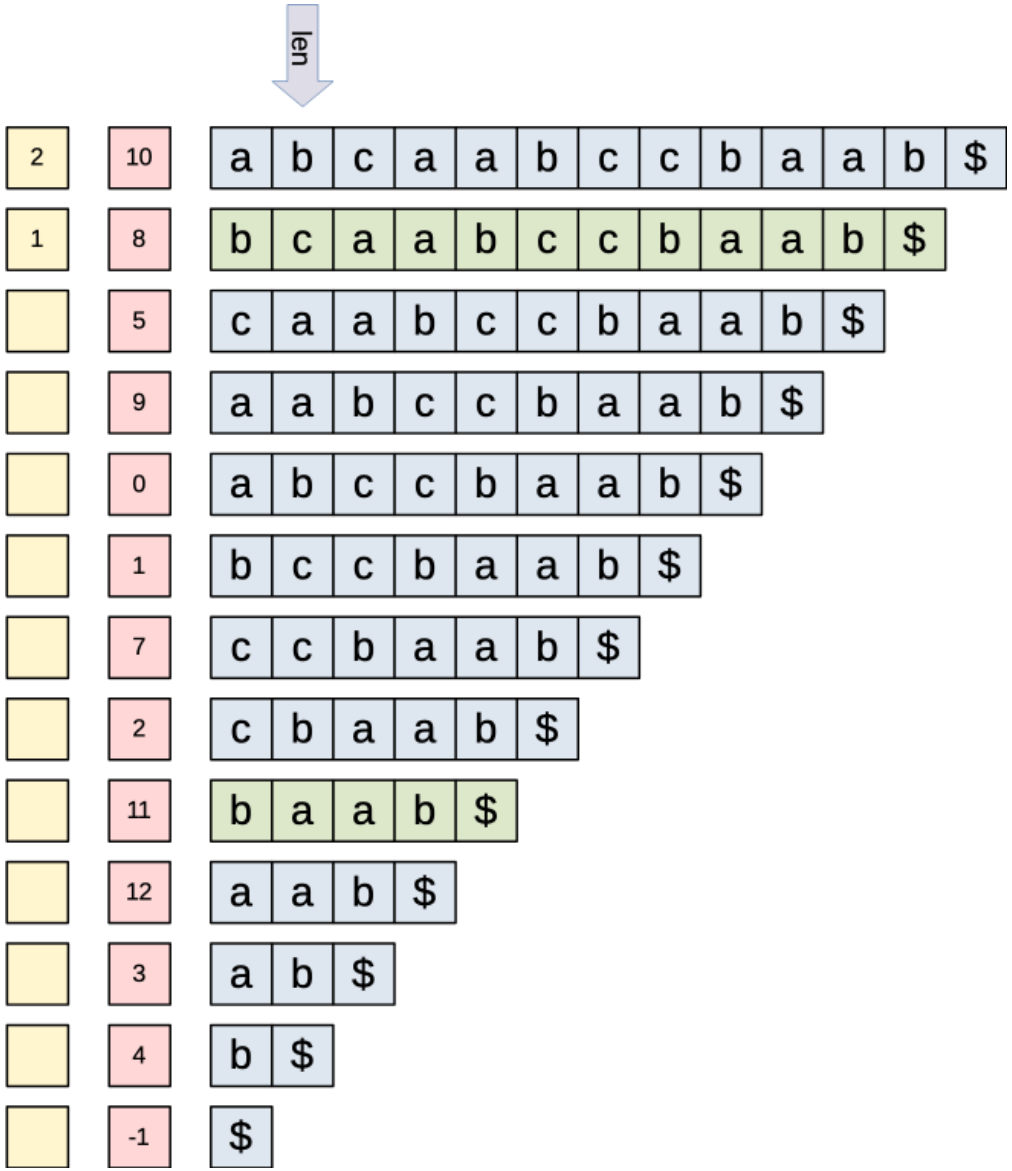
- For the next suffix, we can back up let by 1.
  - This is a clever trick.
  - It keeps us from having to compare the same strings over and over.




I'm the predecessor for the string at offset 1.

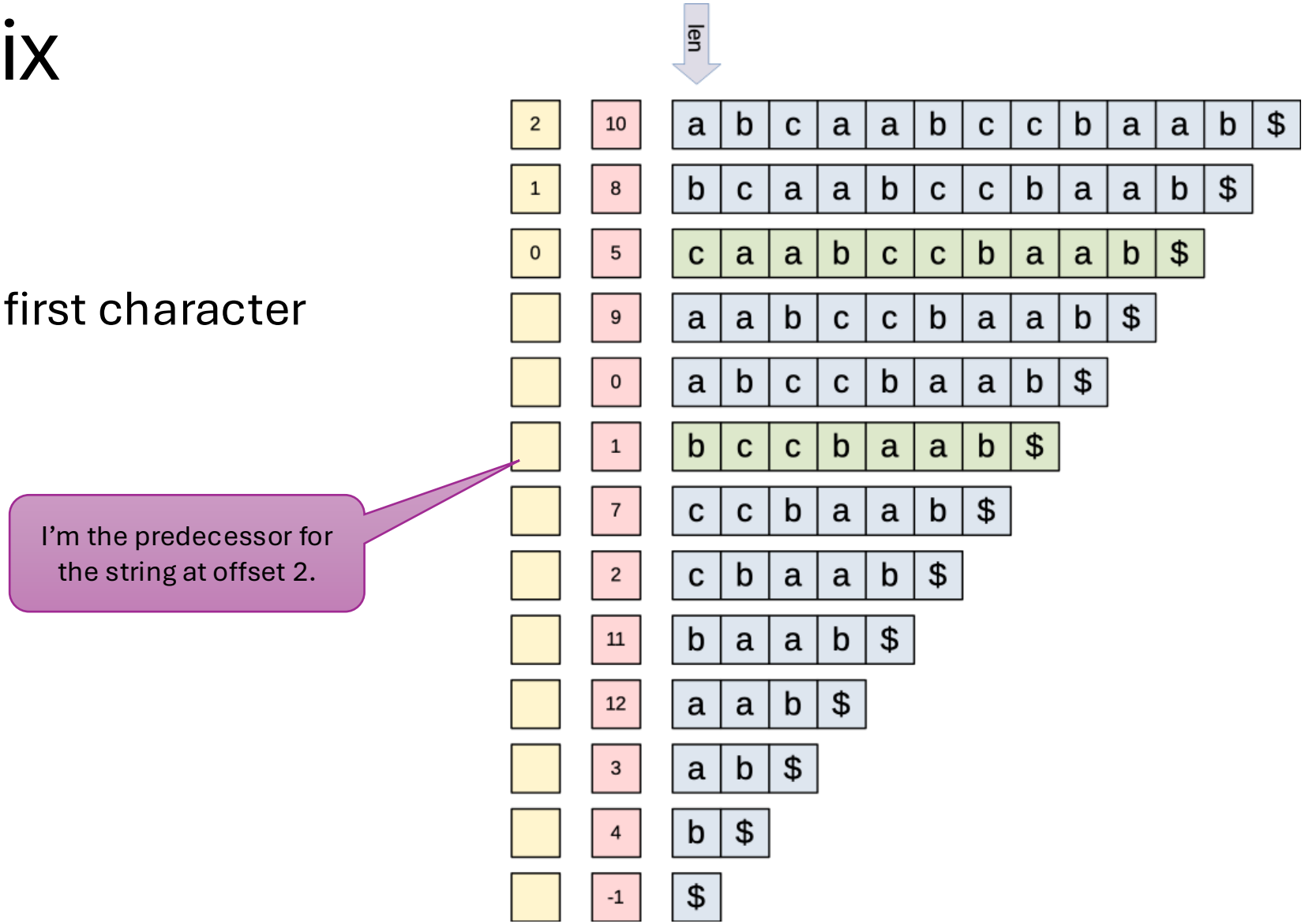
# Computing the Longest Common Prefix

- For each suffix
  - Back up one
  - Move ahead to the first character that disagrees.



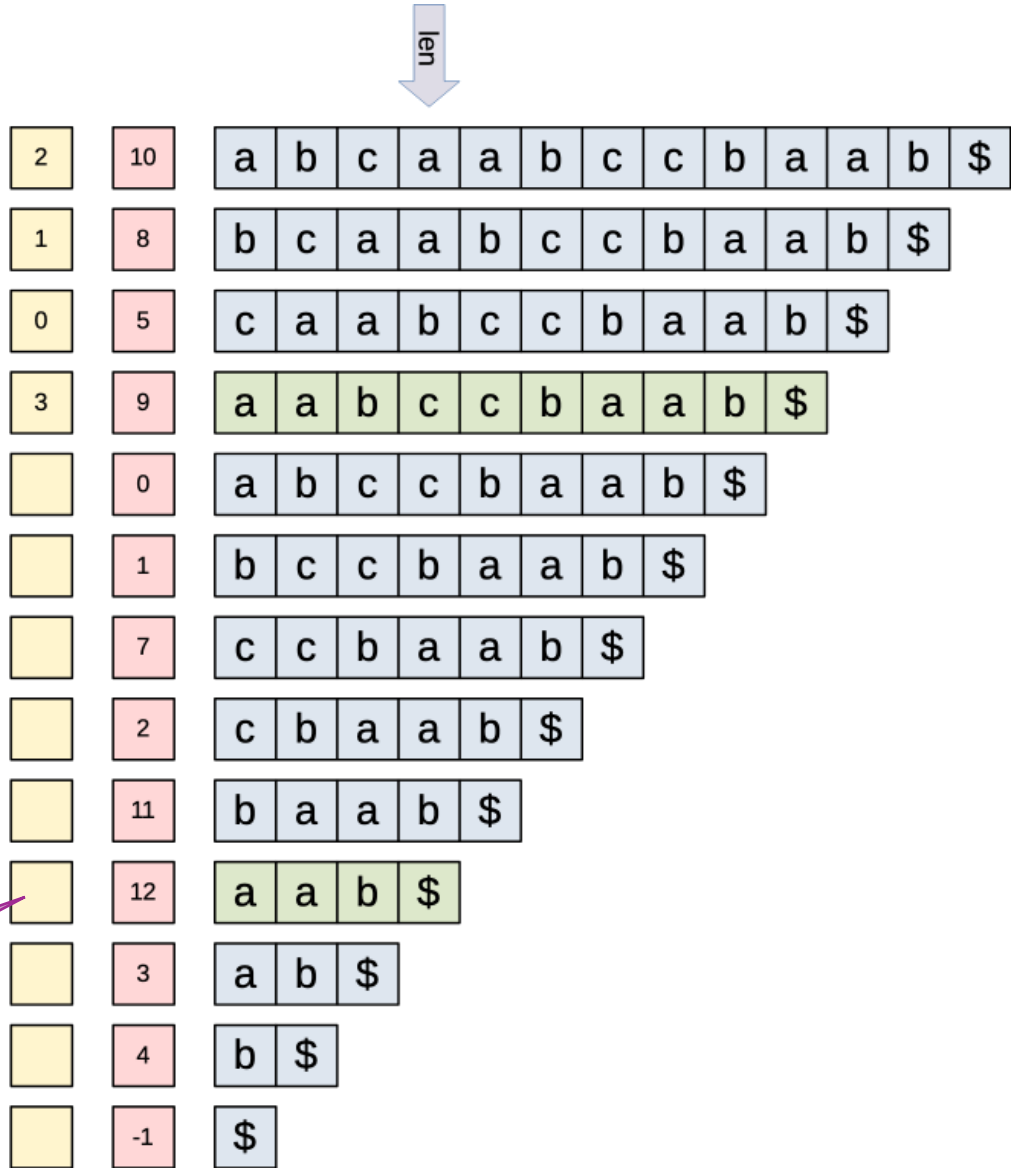
# Computing the Longest Common Prefix

- For each suffix
  - Back up one
  - Move ahead to the first character that disagrees.



# Computing the Longest Common Prefix

- For each suffix
  - Back up one
  - Move ahead to the first character that disagrees.

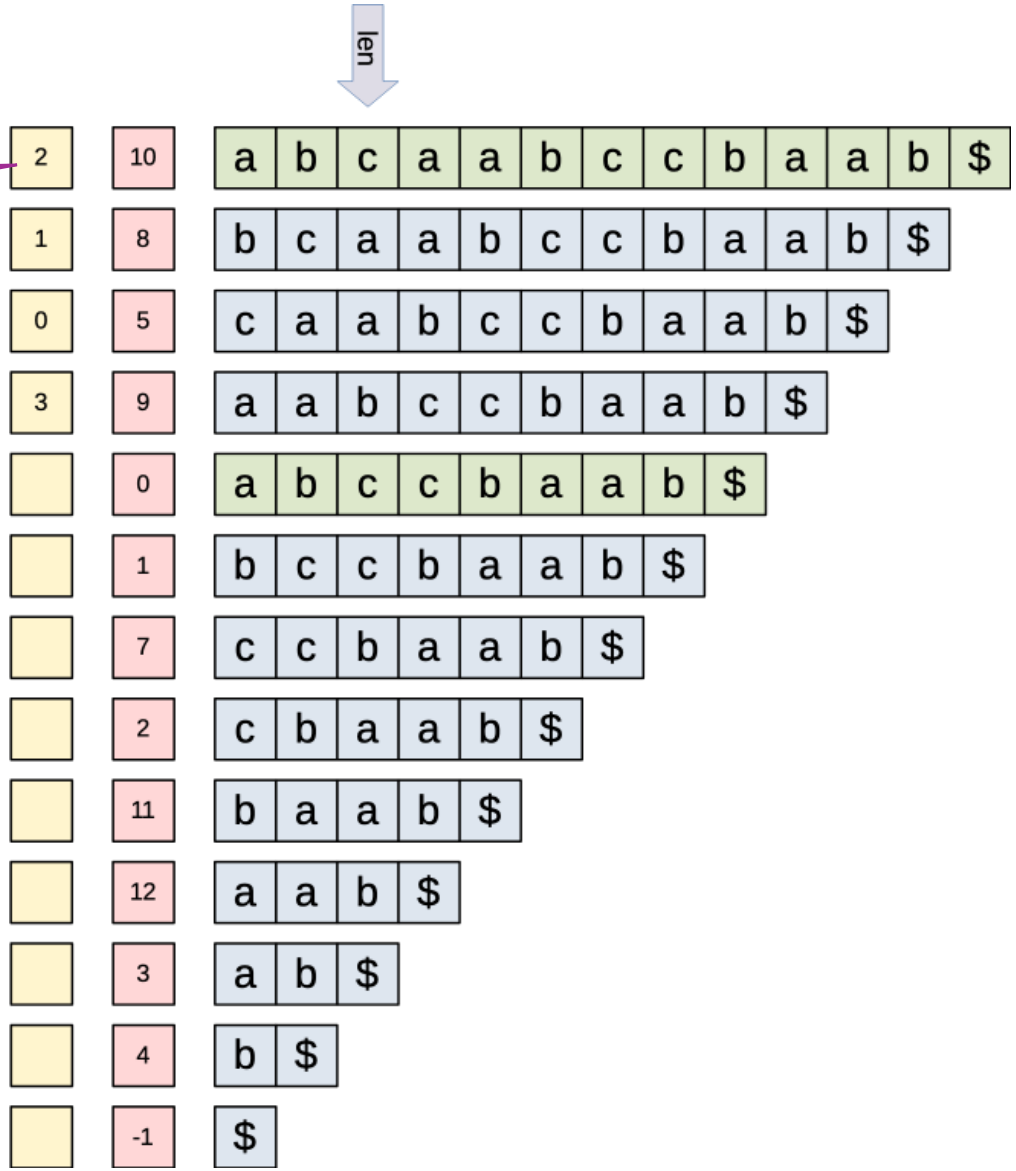



I'm the predecessor for the string at offset 3.

# Computing the Longest Common Prefix

- For each suffix
  - Back up one
  - Move ahead to the first character that disagrees.

I'm the predecessor for the string at offset 4.



# Computing the Longest Common Prefix

- For each suffix
  - Back up one
  - Move ahead to the first character that disagrees.

I'm the predecessor for the string at offset 4.

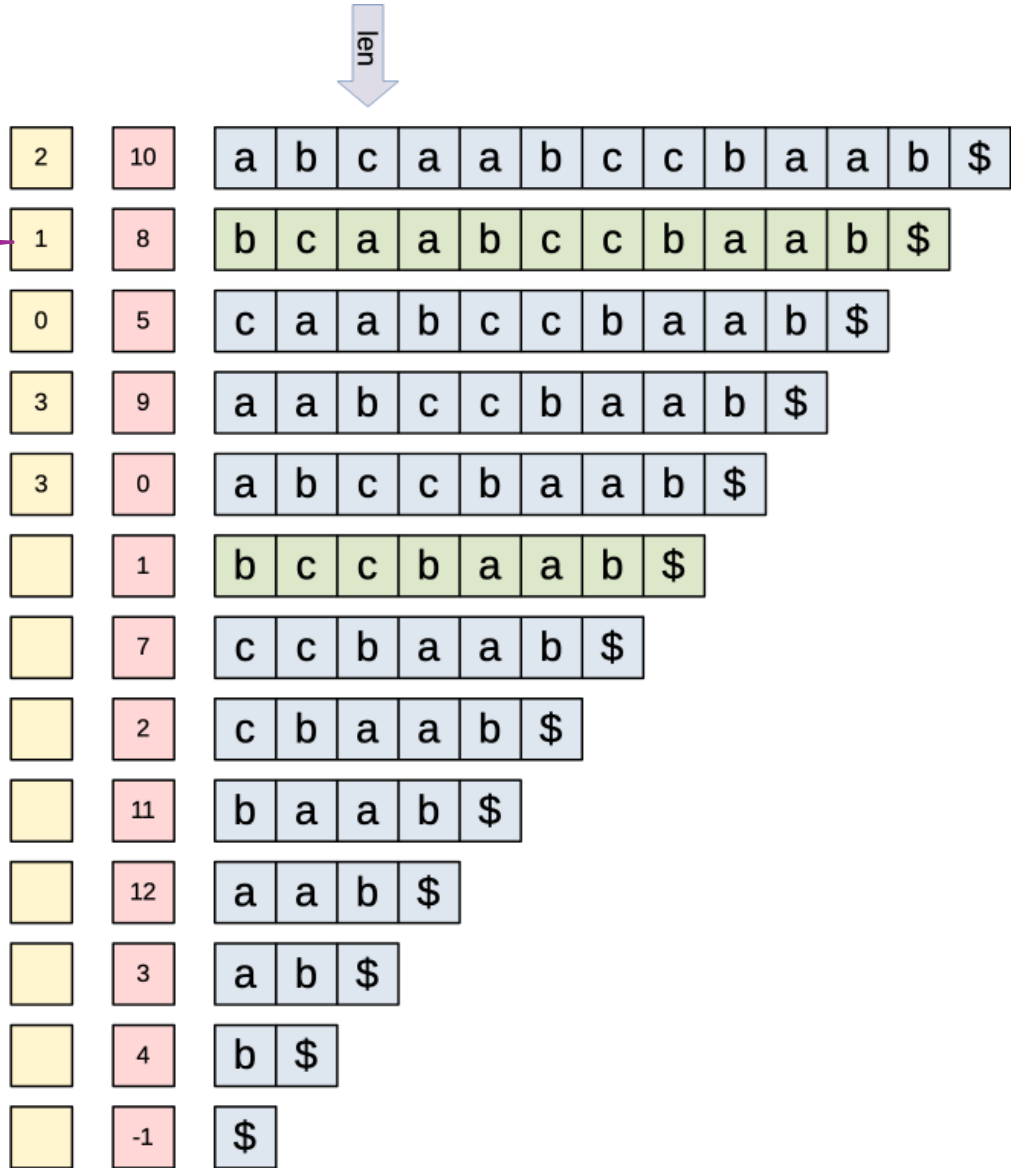
len

| | | |
|---|---|---|
| 2 | 10 | a b c a a b c c b a a b $ |
| 1 | 8 | b c a a b c c b a a b $ |
| 0 | 5 | c a a b c c b a a b $ |
| 3 | 9 | a a b c c b a a b $ |
| 3 | 0 | a b c c b a a b $ |
| | 1 | b c c b a a b $ |
| | 7 | c c b a a b $ |
| | 2 | c b a a b $ |
| | 11 | b a a b $ |
| | 12 | a a b $ |
| | 3 | a b $ |
| | 4 | b $ |
| | -1 | $ |

# Computing the Longest Common Prefix

- For each suffix
  - Back up one
  - Move ahead to the first character that disagrees.

I'm the predecessor for the string at offset 5.



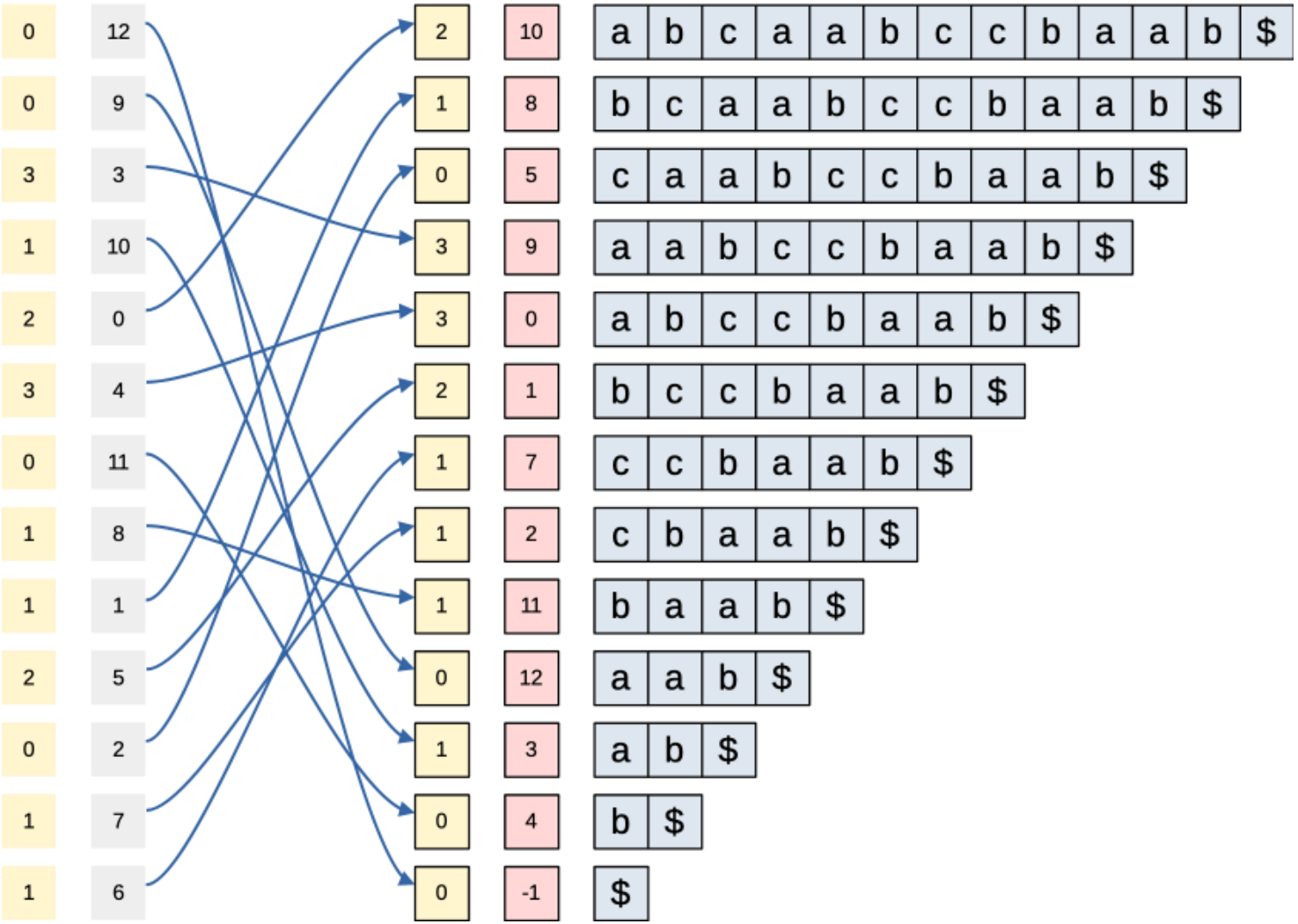
# Computing the Longest Common Prefix

- For each suffix
  - Back up one
  - Move ahead to the first character that disagrees.

| | | |
|---|---|---|
| 2 | 10 | a b c a a b c c b a a b $ |
| 1 | 8 | b c a a b c c b a a b $ |
| 0 | 5 | c a a b c c b a a b $ |
| 3 | 9 | a a b c c b a a b $ |
| 3 | 0 | a b c c b a a b $ |
| 2 | 1 | b c c b a a b $ |
| 1 | 7 | c c b a a b $ |
| 1 | 2 | c b a a b $ |
| 1 | 11 | b a a b $ |
| 0 | 12 | a a b $ |
| 1 | 3 | a b $ |
| 0 | 4 | b $ |
| 0 | -1 | $ |

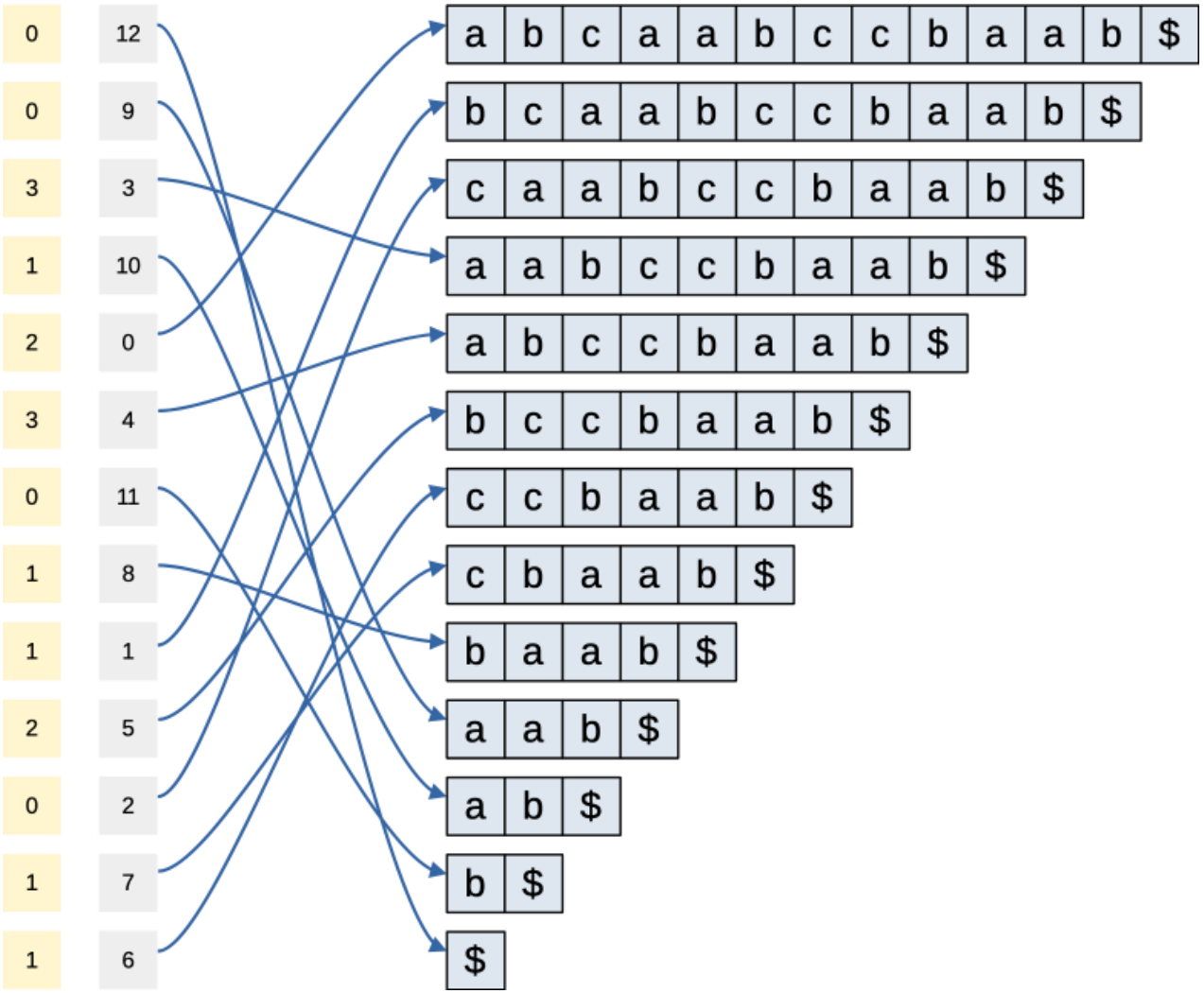
# Computing the Longest Common Prefix

- We can store these lengths along with the suffix array.



# Suffix array and Longest Common Prefix

- These are the results for the suffix array
- And longest common prefix



# Implementing LCP

```
void makeLCP( string const &str, vi const &sa, vi &lcp ) {
  int n = str.length();
  vector< int > phi( n + 1 );

  phi[ sa[0] ] = -1;
  for ( int i = 1; i <= n; i++ )
    phi[ sa[i] ] = sa[ i - 1 ];
```

Compute offset that precedes each suffix.

# Implementing LCP

```
  int len = 0;
  vi plcp( n + 1 );
  for ( int i = 0; i <= n; i++) {
    if ( phi[i] == -1 ) {
      plcp[ i ] = 0;
    } else {
      while ( str[ i + len ] == str[ phi[ i ] + len ] )
        len++;

      plcp[ i ] = len;

      if ( len > 0 )
        len -=1;
    }
  }

  lcp.assign( n + 1, 0 );
  for ( int i = 0; i <= n; i++ )
    lcp[ i ] = plcp[ sa[ i ] ];
}
```

For each suffix.

Move forward as characters match.

Record the length of the match.

Back up at most one character

Store in the same order as the suffix array.